INDIVIDUAL THESIS

AI-DRIVEN SMART INVENTORY AND STOCK FLOW OPTIMIZATION

An Individual Component of the Smart Omni-Retail System

W.M METHSARA

(IT22244598)

Individual component: AI-Driven Smart Inventory and Stock Flow Optimization

Group ID: R26-IT-060

Report submitted in partial fulfilment of the requirements for the

Bsc(Hons) In INFORMATION TECHNOLOGY

INFORMATION TECHNOLOGY

Department of Computing

Sri Lanka Institute of Information Technology

Sri Lanka

OCTOBER / 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: W.M. Methsara

Student ID: IT22244598

Signature: _____ Date: _____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Dr. Dinuka Wijendra		
Co-supervisor: Ms.Jenny Kishara		

ABSTRACT

Multi-branch fashion retailers often face shortages at one branch while another holds surplus stock. Managers typically lack forward-looking demand visibility, a systematic way to choose a source branch, and clear reasons behind software-generated recommendations, and existing studies tend to treat forecasting, transshipment and explanation separately. This research aimed to design, implement and evaluate an integrated inventory decision-support component of the Smart Omni-Retail System, in a retail context inspired by Cool Planet (Pvt) Ltd, Sri Lanka. Following PP1 panel feedback, a synthetic five-year dataset (2021-2025) covering eight stores and about 1,000 products was prepared. An XGBoost regressor forecasts daily, monthly and quarterly demand from lag, rolling, price, promotion, calendar and weather features, and the forecasting service also draws live conditions from the Open-Meteo API at runtime. Forecasts and current stock held in a MongoDB operational database feed an inventory intelligence module that classifies items as shortage, healthy or overstock, and a deterministic engine ranks source stores by coverage, remaining safety stock, distance, estimated logistics cost, risk and confidence. The Groq API explains the engine's results but never selects sources or calculates values, and explanations are cached in MongoDB with a local deterministic fallback. A controlled movement workflow synchronizes inventory and records transactions, and a manager chatbot answers factual questions locally and open-ended questions through the same Groq API. On the evaluation data, the daily model achieved an MAE of 0.5418, an RMSE of 0.9436 and an R^2 of 0.9497, and the monthly model an MAE of 14.213, an RMSE of 26.306 and an R^2 of 0.9353. Scenario testing verified the end-to-end workflow, including duplicate-execution prevention. Because the data is synthetic and stakeholder validation is pending, the results indicate feasibility, not real-world accuracy or financial benefit. My contribution is the forecasting, inventory intelligence, recommendation, explainable AI, workflow and chatbot components.

Keywords: Inventory optimization, demand forecasting, XGBoost, explainable AI, inter-store stock transfer

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my research supervisor, Dr. Dinuka Wijendra and co-supervisor, Jenny Kishara, for their valuable guidance, continuous support, and constructive feedback throughout this research project. Their academic advice and recommendations greatly contributed to the successful development of this research and the preparation of this thesis.

I am also grateful to the academic staff of the Sri Lanka Institute of Information Technology (SLIIT) for providing the knowledge, guidance, and academic environment necessary to undertake this research.

I would like to extend my appreciation to the individuals with experience in inventory management, merchandise planning, information technology, sales, and retail operations who generously shared their knowledge, practical experiences, and valuable feedback. Their insights helped me better understand real-world retail challenges and improve the design and functionality of the proposed system.

My sincere thanks also go to my fellow research group members, J.A.R.K. Wijetunga, T.A Silva, L.A.D.A.S Perera for their collaboration, support, and contributions to the overall Smart Omni-Retail System, particularly during the integration of its different components.

Finally, I would like to express my heartfelt gratitude to my family and friends for their continuous encouragement, patience, understanding, and support throughout my academic journey and the completion of this research.

TABLE OF CONTENTS

DECLARATION AND SUPERVISOR CERTIFICATION i
ABSTRACT ii
ACKNOWLEDGEMENTS iii
Table of Contents iv
LIST OF FIGURES viii
LIST OF TABLES ix
LIST OF ABBREVIATIONS x
1. INTRODUCTION 1
1.1. Overall Project Context 1
1.2. Individual Component Context 2
1.3. Problem Statement 3
1.4. Research Gap 4
1.5. Aim and Objectives 5
1.6. Research Questions or Hypotheses 6
1.7. Scope, Limitations and Assumptions 6
1.8. Individual Contribution 7
1.9. Thesis Structure 8
2. LITERATURE REVIEW AND RELATED WORK 10
2.1. Review Method and Scope 10
2.2. Domain and Theoretical Foundations 11
2.2.1. Demand forecasting in retail 11
2.2.2. Gradient-boosted trees and XGBoost 12
2.2.3. Forecast error metrics and validation 12
2.2.4. Inventory concepts 12
2.2.5. Explainability and language models 13
2.3. Related Research 14
2.3.1. Machine learning for retail sales forecasting 14
2.3.2. Interpretable demand modelling 15
2.3.3. Lateral transshipment and stock transfers 15
2.3.4. Explainable AI and language-model explanations 16
2.3.5. Conversational decision support 16
2.4. Existing Systems, Products and Techniques 17
2.5. Gap Synthesis and Conceptual Framework 19
3. RESEARCH METHODOLOGY 21
3.1. Research Design 21

3.2. Data, Participants or Study Materials 22
3.2.1. Data source and its limits 22
3.2.2. Dataset attributes 23
3.2.3. Inventory data and the scalability decision 24
3.2.4. Participants 24
3.3. Experimental or Study Procedure 25
3.3.1. Data preparation and feature engineering 25
3.3.2. Model selection and training 26
3.3.3. Development and verification procedure for the rest of the component 28
3.4. Measures and Evaluation Metrics 29
3.5. Data Analysis 30
3.6. Ethics, Privacy, Security and Responsible AI 30
3.7. Reproducibility and Research Artefacts 32
4. PRODUCT AND SOLUTION DESIGN 33
4.1. Requirements 33
4.2. Component Architecture Within the Complete Solution 35
4.3. Design Alternatives and Decisions 36
4.4. Detailed Design 38
4.4.1. Forecasting service 38
4.4.2. Inventory intelligence 38
4.4.3. Transfer recommendation engine 39
4.4.4. Explainable AI layer 43
4.4.5. Manager chatbot 46
4.4.6. Stock movement workflow 47
4.4.7. Data model 48
4.5. Standards and Professional Practice 50
4.6. Cross-Component Integration Boundaries 51
5. IMPLEMENTATION, INTEGRATION AND TESTING 52
5.1. Development Environment and Technology Stack 52
5.2. Component Implementation 56
5.2.1. Data and forecasting 56
5.2.2. Inventory intelligence and recommendation 57
5.2.3. Explanation generation and caching 58
5.2.4. Movement workflow and inventory synchronization 59
5.2.5. Manager chatbot 60
5.2.6. Backend module summary 61
5.3. Integration with the Other Components 63

5.3.1. Campaign Feasibility Request and Response Contract 66
5.3.2. Planned End-to-End Verification Scenarios 67
5.4. Verification and Software Testing 68
5.5. Deployment and Operation 70
6. RESULTS AND VALIDATION 72
6.1. Research Results 72
6.1.1. Forecasting performance 72
6.2. Product and Technical Results 74
6.3. User or Stakeholder Validation 79
6.4. Integration Results 81
6.5. Summary of Findings 82
7. DISCUSSION 85
7.1. Interpretation of Findings 85
7.2. Comparison with Related Work 86
7.3. Research Contribution 87
7.4. Product Contribution 88
7.5. Limitations and Threats to Validity 88
7.6. Implications 89
8. COMMERCIALISATION AND SUSTAINABILITY 91
8.1. Value Proposition and Target Users 91
8.2. Market and Competitive Position 91
8.3. Business or Sustainability Model 92
8.4. Scalability, Intellectual Property and Compliance 92
8.5. Contribution to the Sustainable Development Goals 93
9. CONCLUSION AND FUTURE DIRECTIONS 95
9.1. Conclusion 95
9.2. Achievement of Objectives 96
9.3. Individual Research and Product Contributions 96
9.4. Future Directions 97
REFERENCES 99
GLOSSARY 102
APPENDICES 104
Appendix A Extended Results and Research Instruments 105
A.1 Store distance data 105
A.2 Full test-case record 107
A.3 Model Configuration and Technical Specifications 108
A.4 Selected Technical Specifications 108

Appendix B Academic Integrity and AI Use Disclosure 109
B.1 Declaration statement..... 109

LIST OF FIGURES

Figure 1.1: Overall Smart Omni-Retail System context 2
Figure 1.2: Individual component scope within the overall solution 3
Figure 2.1: Conceptual framework for AI-driven inventory optimization 20
Figure 3.1: Research methodology workflow 21
Figure 3.2: Five-year dataset preparation and feature-engineering pipeline 26
Figure 3.3: Machine-learning training and evaluation workflow 28
Figure 4.1: Overall architecture of the individual component 35
Figure 4.2: FastAPI layered backend architecture 36
Figure 4.3: Inventory intelligence workflow 39
Figure 4.4: Transfer recommendation decision workflow 41
Figure 4.5: Explainable AI architecture 46
Figure 4.6: Manager chatbot architecture 47
Figure 4.7: Stock movement state and lifecycle diagram 48
Figure 4.8: MongoDB data model and key collection relationships 49
Figure 5.1: End-to-end stock transfer sequence diagram 60
Figure 5.2: Integration with the Smart Mirror and other group components 64
Figure 6.1: Daily forecasting performance visualization 73
Figure 6.2: Monthly forecasting performance visualization 73
Figure 6.3: Example recommendation and alternative-store comparison 76
Figure 6.4: Actual Application Output Showing the Explainable Stock Transfer Recommendation, Inventory Impact, and Logistics Evaluation. 77
Figure 6.5: Example recommendation and alternative-store comparison 78

LIST OF TABLES

Table 1.1: Components of the Smart Omni-Retail System 2
Table 1.2: Research objectives 5
Table 1.3: Research questions 6
Table 2.1: Indicative comparison of approaches 19
Table 3.1: Stores in the study 23
Table 3.2: Attributes of the historical sales dataset 23
Table 3.3: Feature variables used by the daily forecasting model 26
Table 3.4: Candidate forecasting techniques considered 27
Table 3.5: Forecast evaluation metrics 29
Table 4.1: Functional requirements 33
Table 4.2: Non-functional requirements 34
Table 4.3: Key design decisions and alternatives 37
Table 4.4: Recommendation criteria used by the deterministic engine 40
Table 4.5: Stock movement statuses 48
Table 4.6: Key MongoDB collections 49
Table 5.1: Technology stack 54
Table 5.2: Backend module summary 62
Table 5.3: Test-case summary 69
Table 6.1: Forecast evaluation results 72
Table 6.2: Stakeholder consultation and evidence status 80
Table 7.1: Summary of research contributions and strength of evidence 87
Table 7.2: Limitations and their effects 89
Table 8.1: Relationship to the Sustainable Development Goals 93
Table 9.1: Objective-achievement matrix 96
Table 9.2: Future work 97
Table A.1: Store-to-store distances used by the engine 105
Table A.2: Detailed test-case record 107
Table A.3: Selected Technical Specifications 108
Table B.1: AI use disclosure 109

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
ARIMA	Autoregressive Integrated Moving Average
ERP	Enterprise Resource Planning
FR	Functional Requirement
IEEE	Institute of Electrical and Electronics Engineers
IP	Intellectual Property
LLM	Large Language Model
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
ML	Machine Learning
NFR	Non-Functional Requirement
NoSQL	Not only Structured Query Language
PDPA	Personal Data Protection Act
POS	Point of Sale
PP1, PP2, PP3	Progress Presentations 1, 2 and 3
R^2	Coefficient of Determination
RMSE	Root Mean Squared Error
ROI	Return on Investment
SDG	Sustainable Development Goal
SHAP	SHapley Additive exPlanations
SLIIT	Sri Lanka Institute of Information Technology
UI	User Interface
XAI	Explainable Artificial Intelligence
XGBoost	Extreme Gradient Boosting

1. INTRODUCTION

1.1. Overall Project Context

Fashion retail in Sri Lanka is increasingly multi-channel. A typical mid-sized retailer runs several physical showrooms, keeps a growing product catalogue, and is under pressure to make better use of digital tools without the budget of a global chain. The group research project, titled the Smart Omni-Retail System, was developed in a retail setting inspired by Cool Planet (Pvt) Ltd, a Sri Lankan fashion retailer that sells brands such as Emerald, Hustle, Miss Modano, Modano, King Street and Libera through multiple branches. The retailer is used as a reference context for the design of the system. No confidential transaction records from the company were used in this work, and the historical dataset described in Chapter 3 was synthetically generated to represent realistic retail behaviour.

The Smart Omni-Retail System is made up of four integrated components, each developed by a different group member. Figure 1.1 shows how they relate to one another, and Table 1.1 lists the components. My own component is the AI-driven inventory and stock-flow optimization module. It supplies demand forecasts and stock-transfer decisions that the other components can consume, and it shares product and store master data with them.

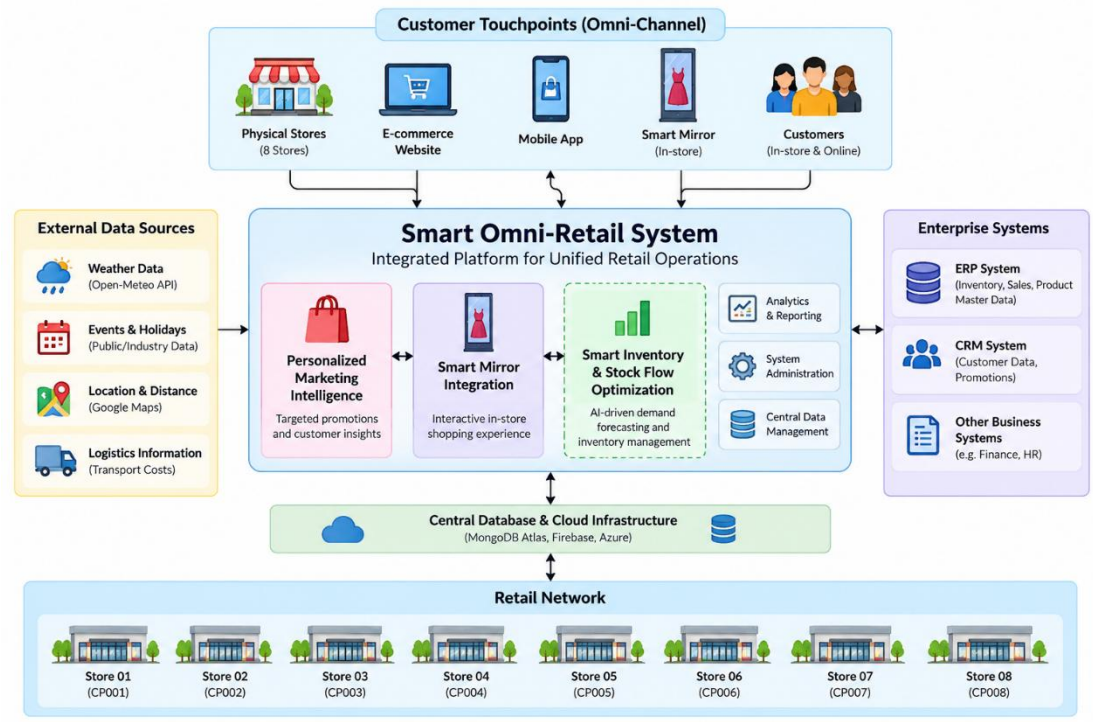


Figure 1.1: Overall Smart Omni-Retail System context

Table 1.1: Components of the Smart Omni-Retail System

No.	Component	Member	Student ID
1	AI-Driven Smart Inventory and Stock-Flow Optimization	W.M Methsara	IT22244598
2	AI-Driven Personalized Marketing Intelligence	WIJETUNGA J A R K	IT22146588
3	AI-Based Virtual Try-On and Size Recommendation (Smart Mirror)	Silva T. A	IT22284952
4	GPS-Based Customer Behaviour Tracking with Staff Assistance	L.A.D.A.S. Perera	IT22243812

1.2. Individual Component Context

The individual component addressed in this thesis is titled AI-Driven Smart Inventory and Stock Flow Optimization. It is meant to support the people who decide how stock is distributed across a retail network: inventory managers, store managers and operations staff. Instead of treating forecasting, stock monitoring, transfer planning and reporting as separate tasks, the component brings them together in one decision-support workflow.

In practical terms the component does six things. It forecasts demand at daily, monthly and quarterly levels. It assesses inventory health by comparing forecast demand with the stock on hand. It detects shortages and overstock and estimates when stock will run out. It recommends stock transfers between branches and ranks the possible source stores. It explains each recommendation in plain language, and it lets managers approve, reject, cancel and execute transfers through a controlled workflow that keeps inventory records consistent. A manager chatbot sits on top of these features as a conversational interface. Figure 1.2 shows where these functions sit within the wider system.

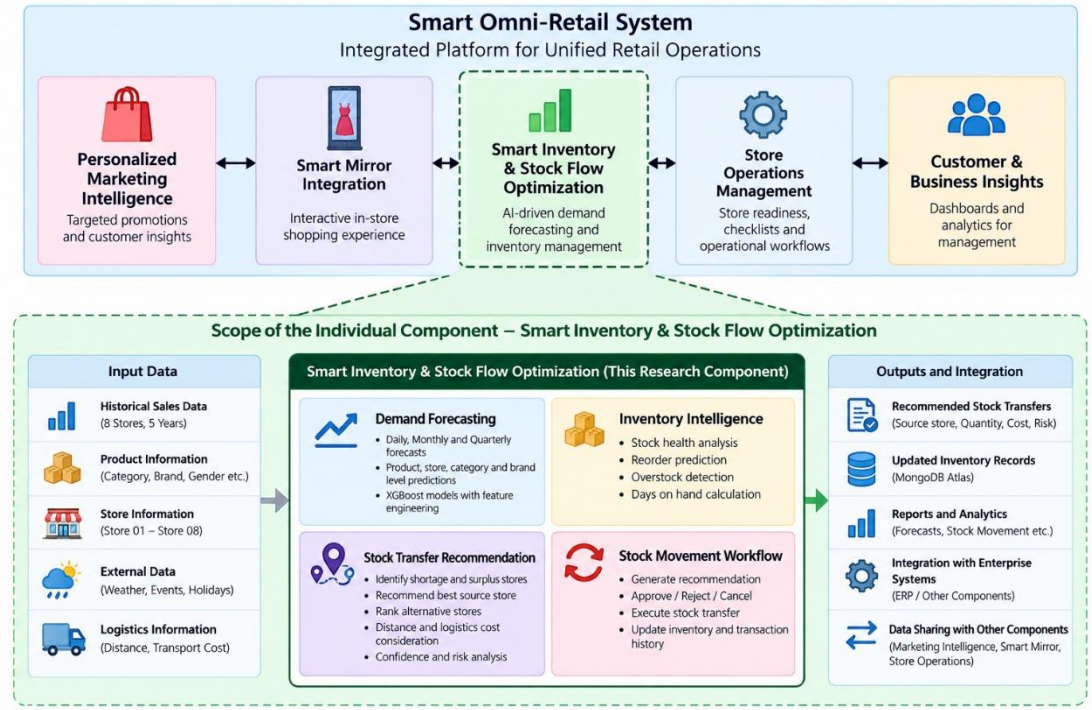


Figure 1.2: Individual component scope within the overall solution

1.3. Problem Statement

Multi-branch fashion retailers face a recurring stock imbalance. One showroom runs out of a size or style while another holds surplus units of the same product that are moving slowly. Managers usually find out about this late, and the response is often a phone call or a spreadsheet exercise rather than a systematic decision. The consequence is lost sales at the branch that ran short and money tied up in stock at the branch that overstocked.

Several difficulties make this hard to solve by hand. Future demand for a product at a particular branch is not visible, so managers rely on recent experience. Stock transfers are frequently reactive, triggered after a shortage has already occurred. When a shortage does occur, choosing the best source branch involves more than finding the store with the most stock. The source must be able to cover the shortage without being left short itself, and the distance and cost of moving goods matter as well. Finally, when a software tool does suggest an action, managers are unlikely to trust it if they cannot see why it was suggested. Fildes et al. describe how retailers need forecasts at strategic, tactical and operational levels [1], which suggests that a forecast is only useful once it is connected to an operational decision.

The problem addressed by this thesis can therefore be stated as follows. Retail managers need a way to anticipate demand at branch level, identify shortage and surplus situations early, choose a sensible source branch for stock transfers, and understand and control the resulting decisions, using a single integrated tool rather than a set of disconnected reports.

1.4. Research Gap

The literature on retail demand forecasting is large [1], [2], and lateral transshipment, the movement of stock between locations at the same level of the supply chain, has been studied for decades [3]. Machine-learning forecasters such as gradient-boosted trees perform well on tabular retail data [4], [5]. However, in the sources reviewed for this work, forecasting and transshipment are typically studied separately. Forecasting studies stop at the prediction, and transshipment models tend to assume demand distributions rather than consume forecasts from a deployed model. Work on fashion retail transshipment does exist [6], but it is formulated as a mathematical optimization problem and is not concerned with how a manager would interact with the resulting decision.

A second gap concerns explanation. Explainable AI research has produced feature-attribution methods such as SHAP [7], and surveys stress that explanations should be tailored to their audience [8]. Most of this work explains a model's prediction. In an inventory setting, the manager's question is different: why was this store chosen to

supply the stock, and why not the other one? That is a question about a business decision made by a rule-based engine, not about a model output. Large language models can write fluent explanations, but they are known to produce statements that are not grounded in the input [9], which is unacceptable if the explanation is supposed to describe stock quantities and costs.

The gap addressed here is therefore the lack of an integrated, manager-facing inventory decision-support component that (a) links demand forecasts to inventory health and transfer recommendations, (b) keeps the transfer decision deterministic and auditable, (c) uses a language model only to explain decisions whose facts have already been computed, and (d) manages the execution of the transfer so that inventory records remain consistent. The literature reviewed in Chapter 2 does not describe this exact combination, although individual elements exist. This statement is limited to the sources I reviewed and is not a claim that no such system exists.

1.5. Aim and Objectives

The aim of this research is to design, implement and evaluate an integrated AI-driven inventory decision-support component for a multi-branch fashion retailer that forecasts demand, assesses stock health, recommends explainable inter-store stock transfers, and manages the transfer lifecycle.

The objectives derived from this aim are listed in Table 1.2. They are referred to as O1 to O8 in later chapters, and Section 9.2 reports how far each one was achieved.

Table 1.2: Research objectives

ID	Objective	Addressed in
O1	Prepare a five-year multi-store historical sales dataset that captures seasonality, trend, promotions, weather and calendar effects	Chapter 3
O2	Develop and evaluate an XGBoost-based forecasting model for daily, monthly and quarterly demand	Chapters 3, 5, 6
O3	Build an inventory intelligence module that classifies stock as SHORTAGE, HEALTHY or OVERSTOCK using forecast demand and current stock	Chapters 4, 5
O4	Design a deterministic recommendation engine that ranks source stores using stock, distance, cost and risk criteria	Chapters 4, 5, 6
O5	Implement an explainable AI layer in which a language model explains, but never makes, recommendation decisions	Chapters 4, 5, 6

ID	Objective	Addressed in
O6	Implement a controlled stock movement workflow that keeps inventory and transaction records consistent	Chapters 4, 5, 6
O7	Provide a manager chatbot that answers factual questions locally and open-ended questions through conversational AI	Chapters 4, 5
O8	Verify the component through functional testing and integrate it with other group components	Chapters 5, 6

1.6. Research Questions or Hypotheses

Four research questions guide the work. They are listed in Table 1.3 together with the evidence that is used to answer each one. I framed them as questions and not as formal statistical hypotheses because the evaluation relies on forecast error metrics and functional verification. No hypothesis tests were performed.

Table 1.3: Research questions

ID	Research question	Evidence
RQ1	How accurately can an XGBoost model forecast daily and monthly product demand at branch level on the five-year retail dataset?	MAE, RMSE and R^2 (Section 6.1)
RQ2	Can current stock and forecast demand be combined into a useful classification of shortage, healthy and overstock situations?	Inventory intelligence outputs and testing (Sections 5.4, 6.2)
RQ3	Can a deterministic engine recommend and rank source stores in a way that considers coverage, safety stock, distance, cost and risk, and can a language model explain those decisions without altering them?	Recommendation examples, explanation outputs, workflow testing (Sections 6.2, 6.3)
RQ4	Can a controlled approval and execution workflow keep inventory records consistent when transfers are executed?	Movement lifecycle tests and data integrity checks (Sections 5.4, 6.2)

1.7. Scope, Limitations and Assumptions

The component covers demand forecasting, inventory intelligence, transfer recommendation, explainable AI, the stock movement workflow and the manager chatbot for the eight-store, roughly 1,000-product retail scenario described in Chapter 3. It is delivered as a Flutter Web application backed by a FastAPI service, with Firebase Authentication for sign-in and MongoDB as the application database.

Several boundaries apply. The historical data is synthetic, generated to represent realistic behaviour, so the forecasting results describe how well the model recovers

patterns in that dataset, not how it would perform on a real enterprise sales history. There is no live connection to an ERP or point-of-sale system. Logistics cost is a simplified estimate. The explainable AI and chatbot features depend on external services that have availability and quota limits. The component was not deployed across real showrooms, and its business benefit in monetary terms was not measured. Section 7.5 discusses these limitations in more detail.

I made the following assumptions. Current stock levels held in the inventory collection are taken to be accurate. Distances between stores are treated as fixed values. A transfer is executed in a single step from the manager's point of view, even though a real movement takes time. The product image support described in Section 5.3 uses representative images, not product-specific photographs.

1.8. Individual Contribution

This research represents my individual contribution to the group research project, *Smart Omni-Retail System*. My primary responsibility was the design, development, implementation, and evaluation of the AI-Driven Smart Inventory and Stock Flow Optimization component.

I independently developed the frontend interfaces and associated backend functionality for all seven Smart Inventory modules: Dashboard, Demand Forecasting, Inventory Intelligence, Stock Optimization, Stock Movements, Analytics, and Manager Assistant. The frontend was developed using Flutter Web, while the backend services were implemented using Python and FastAPI.

My technical contributions included the development of the daily and monthly demand forecasting pipelines using XGBoost, inventory health analysis, reorder and overstock identification, a deterministic inter-store stock transfer recommendation engine, and an explainable AI layer for interpreting forecasting and inventory decisions. I also implemented the stock movement approval and execution workflow, including source and destination inventory updates, transaction logging, and the Manager Assistant chatbot. Furthermore, I developed the backend services and MongoDB data structures required to support these functionalities.

I contributed to the integration of the Smart Inventory component with other components of the Smart Omni-Retail System. For the Smart Mirror component, my contribution involved providing product information, image metadata, image URLs, and inventory availability information through the shared system infrastructure. For the Personalized Marketing Intelligence component, I implemented the sharing of inventory-based promotional opportunities. The reverse workflow, through which Marketing requests inventory feasibility checks for proposed campaigns, was partially implemented at the time of writing.

Certain infrastructure and integration elements were developed collaboratively within the research group. These included the shared authentication infrastructure and the MongoDB database used to exchange common product and store information between components. The Smart Mirror interface, marketing intelligence models, virtual try-on functionality, and customer tracking features were developed by other group members and are not claimed as part of my individual implementation.

Finally, I conducted functional testing and verification of the Smart Inventory modules, including forecasting services, inventory analysis, recommendation generation, stock movement processing, and integration-related functionality. The implementation and evaluation of these contributions are discussed in the subsequent chapters of this thesis.

1.9. Thesis Structure

The rest of the thesis is organized as follows. Chapter 2 reviews the literature on retail demand forecasting, inventory transshipment, explainable AI and conversational decision support, and ends with the conceptual framework. Chapter 3 describes the research methodology, including the dataset, the forecasting experiments, the evaluation measures and the ethical considerations. Chapter 4 presents the requirements, architecture and detailed design. Chapter 5 covers implementation, integration and testing. Chapter 6 reports the results and the validation evidence that is currently available. Chapter 7 discusses the findings against the literature and states the limitations. Chapter 8 considers commercialisation and sustainability, and Chapter

9 concludes and outlines future work. The references, glossary and appendices follow, including the disclosure of generative AI use required by the university.

2. LITERATURE REVIEW AND RELATED WORK

2.1. Review Method and Scope

This chapter reviews existing research relevant to the AI-Driven Smart Inventory and Stock Flow Optimization component. The literature review focuses on five main areas: retail demand forecasting, gradient-boosted tree models, multi-location inventory management and stock transfers, explainable artificial intelligence, and large language model-based explanations and conversational decision support. Cloud-based services and NoSQL databases, including MongoDB [10], are also briefly considered where they influenced system design decisions.

A targeted literature review approach was adopted rather than a systematic literature review. Relevant academic publications were identified through Google Scholar and IEEE Xplore. The review focused on research related to demand forecasting, machine learning techniques for sales prediction, multi-store inventory optimization, stock transfer recommendations, explainable AI, and intelligent decision-support systems.

Priority was given to peer-reviewed journal articles and conference papers published between 2020 and 2026. Earlier publications were also considered where they introduced foundational algorithms, concepts, or evaluation methods relevant to the research.

The literature was examined for its relevance to the research objectives, methodological contributions, and applicability to retail inventory management. Particular attention was given to the strengths and limitations of existing approaches and the opportunities for integrating demand forecasting, inventory intelligence, stock transfer optimization, and explainable AI within a unified system.

The purpose of this review is to establish the theoretical and technical foundation for the proposed solution, identify relevant research gaps, and justify the approaches adopted in the system design and implementation. This chapter presents a focused review of relevant literature and does not claim to be a comprehensive systematic literature review.

2.2. Domain and Theoretical Foundations

2.2.1. Demand forecasting in retail

Retailers use demand forecasting to support operational, tactical, and strategic decision-making. Long-term forecasts assist with strategic decisions such as store expansion, resource allocation, and business planning, whereas short-term forecasts support replenishment, inventory allocation, and stock movement decisions at individual product and store levels [1].

Forecasting demand at the product-store-day level is particularly challenging because sales quantities may be low, intermittent, and influenced by multiple factors, including pricing, promotional activities, weather conditions, public holidays, and local events. These challenges are especially significant in fashion retail, where demand is affected by changing consumer preferences, seasonal trends, short product life cycles, and variations in product attributes such as size and colour [21].

Swaminathan and Venkitasubramony [21] reviewed demand forecasting approaches for fashion products, highlighting the importance of selecting appropriate forecasting methods for an industry characterised by demand uncertainty and rapidly changing trends. Similarly, Güven and Şimşir [22] investigated demand forecasting in the retail apparel industry using artificial neural networks and support vector machines, incorporating product colour as a forecasting parameter. Their research demonstrates the relevance of product-specific attributes when developing apparel demand prediction models.

These characteristics indicate that fashion retail demand forecasting requires approaches capable of analysing historical sales patterns alongside product characteristics and contextual variables. Machine learning techniques offer a practical approach to modelling relationships between these factors and future demand.

In this research, demand forecasting is implemented using XGBoost-based models to estimate future product demand across multiple retail stores. Historical sales information, temporal features, promotional indicators, weather-related variables, and other engineered features are incorporated into the forecasting process. The resulting predictions support inventory intelligence, shortage identification, overstock

detection, and inter-store stock transfer recommendations within the proposed Smart Inventory system.

2.2.2. Gradient-boosted trees and XGBoost

Gradient boosting builds a prediction as a sum of many small decision trees, each fitted to the errors left by the previous ones. XGBoost is a widely used implementation that adds regularization, handles sparse and missing values, and is engineered to train quickly on large tabular datasets [4]. For the kind of data used in this project, where each row describes a store, product and date with numerical and categorical attributes, tree ensembles have a good empirical record. A benchmark across 45 tabular datasets found that tree-based models, including XGBoost, remained state of the art on medium-sized data even when the compute cost of hyperparameter search was taken into account [5]. This supports, but does not prove, the choice of XGBoost for the present dataset.

2.2.3. Forecast error metrics and validation

Three metrics are used in this thesis. The mean absolute error (MAE) is the average size of the forecast errors in the units of the target, and it is easy to explain to a manager. The root mean squared error (RMSE) squares errors before averaging, so large misses count for more. Chai and Draxler argue that neither metric is universally better and that a combination of metrics is usually needed to assess a model [11]. The coefficient of determination (R^2) compares the model's squared error with that of a model that always predicts the mean, so a value close to 1 means most of the variance in the target is explained. Section 3.4 gives the definitions. Because sales data is ordered in time, validation should respect chronology, so that the model is never evaluated on data that precedes its training period [12].

2.2.4. Inventory concepts

Several fundamental inventory management concepts are used throughout this research to evaluate stock availability and support replenishment and stock movement decisions.

Stock on hand refers to the quantity of inventory currently available at a particular store or storage location. It provides a measure of existing stock availability and serves as a basis for evaluating whether expected customer demand can be satisfied.

Days on hand, used in this research as an estimate of forward-looking stock coverage, indicates the approximate number of days that available inventory can support demand at the expected daily consumption rate. This measure helps identify products that may experience shortages or remain in inventory for extended periods.

The reorder level, also known as the reorder point, represents the inventory threshold at which replenishment should be initiated. Safety stock refers to additional inventory maintained to protect against uncertainty in demand and replenishment lead times. These concepts are widely used in inventory management to reduce the risk of stockouts while balancing inventory availability and holding requirements [23].

A stock shortage occurs when available inventory is insufficient to meet expected demand over a specified planning period. Conversely, overstock refers to inventory quantities exceeding the levels justified by anticipated demand. Both conditions can affect retail operations: shortages may lead to lost sales opportunities, while excessive inventory can increase storage requirements and tie up working capital.

In the proposed Smart Inventory system, these concepts are applied through the Inventory Intelligence module to analyse inventory health, estimate stock coverage, identify potential shortages and excess inventory, and support replenishment and inter-store transfer decisions. The indicators are used for operational decision support and do not represent a claim that the system calculates mathematically optimal inventory policies.

2.2.5. Explainability and language models

Explainable AI aims to make the behaviour of a system understandable to the people who use it. Arrieta et al. emphasise that what counts as a good explanation depends on the audience [8], and Miller, drawing on the social sciences, observes that people tend to ask contrastive questions such as why one option was chosen instead of another [13]. That observation is relevant here, because a manager looking at a transfer recommendation will naturally ask why one store was selected and not another.

Large language models can produce fluent text, but they are prone to hallucination, meaning output that is not supported by the input [9]. One common response is to ground the model in retrieved or supplied facts. Retrieval-augmented generation is one formulation of this idea, in which a generator is conditioned on external knowledge [14]. The explainable AI layer in this project uses a simpler form of grounding: the system supplies a structured set of facts and asks the model to describe only those facts.

2.3. Related Research

2.3.1. Machine learning for retail sales forecasting

Fildes, Ma, and Kolassa [1] review retail forecasting research and practice, identifying challenges associated with promotional activities, product hierarchies, and forecasting demand for new products. Their work demonstrates the importance of selecting forecasting methods that account for the complexity of retail operations and the availability of explanatory variables.

The M5 Accuracy Competition provides further evidence of the effectiveness of machine learning techniques for hierarchical retail sales forecasting. Makridakis, Spiliotis, and Assimakopoulos [2] evaluated forecasting approaches using 42,840 hierarchical sales time series derived from Walmart retail data. The competition results showed that all of the top 50 submissions employed LightGBM, a gradient-boosted decision-tree algorithm, and achieved overall forecasting accuracy improvements exceeding 14% compared with the strongest statistical benchmark. The study also highlighted the importance of explanatory variables, forecast combinations, and appropriate validation strategies.

These findings support the suitability of gradient-boosted tree methods for retail demand forecasting. However, the competition primarily demonstrated the performance of LightGBM and should not be interpreted as direct evidence that XGBoost outperforms alternative forecasting algorithms.

Wellens, Boute, and Udenio [15] investigate tree-based forecasting methods incorporating explanatory variables and examine approaches for simplifying these

models. Their research further supports the practical relevance of tree-based techniques for retail sales prediction.

Based on these findings, the present research adopts XGBoost for daily and monthly demand forecasting, incorporating historical sales, calendar information, promotional indicators, weather-related variables, and engineered temporal features.

While the reviewed forecasting studies primarily evaluate predictive performance, the present research extends the application of forecasting by connecting demand predictions to inventory health assessment, shortage and overstock identification, and inter-store stock transfer recommendations. This integration represents the principal application-level distinction of the proposed Smart Inventory component.

2.3.2. Interpretable demand modelling

Antipov and Pokryshevskaya combined gradient boosting with Shapley values to obtain interpretable demand models from high-dimensional data [16]. Their approach explains a model, showing which inputs drive predictions. This thesis addresses a related but different need. The explanations required by a manager concern a business recommendation, and the recommendation engine is rule-based, not a learned model. Feature attribution for the forecasting model, for example with SHAP [7], remains a possible extension (Section 9.4).

2.3.3. Lateral transshipment and stock transfers

Paterson et al. review inventory models with lateral transshipments, which are stock movements between locations at the same echelon. They distinguish transshipments carried out periodically to redistribute stock from those used reactively when demand cannot be met from stock on hand, and they note that system size, cost structure and service-level definition all influence the best approach [3]. Naderi, Kilic and Dasci present a deterministic model for the transshipment problem of a fast-fashion retailer under capacity constraints [6], which is close to the setting of this thesis in industry, though it is an optimization model and not an interactive decision-support tool.

The transfer engine in this thesis does not attempt to solve an optimal transshipment problem. It scores and ranks candidate sources under transparent rules. This is a deliberate simplification, discussed in Section 4.3.

2.3.4. Explainable AI and language-model explanations

The XAI survey by Arrieta et al. organizes explainability methods and links them to the idea of responsible AI, with accountability and audience at the centre [8]. SHAP assigns each feature an importance value for an individual prediction [7]. These are model-centric methods. For decision-support tools whose logic is a set of explicit business rules, the rules themselves are already inspectable, and the remaining challenge is presenting their outcome in language a manager can absorb quickly. Using a language model for that purpose is attractive, but the hallucination risk documented by Ji et al. [9] means the model should not be trusted to calculate or select anything. This concern is what motivated the architecture in Section 4.4, in which the Groq API [17] receives only facts that the deterministic engine has already computed.

2.3.5. Conversational decision support

Conversational decision-support systems provide an alternative way for users to interact with business information through natural-language questions rather than relying exclusively on conventional dashboards and reporting interfaces. Such systems can make data exploration more accessible to users who do not have specialised database querying or analytical skills.

Quamar et al. [24] present an ontology-driven conversational business intelligence system that enables users to interact with business information through a conversational interface. Their research demonstrates how natural-language interaction can support access to business data and analytical information.

Similarly, Kanburoğlu and Tek [25] review approaches for translating natural-language questions into database queries, including methods based on large language models. Their study identifies both the potential of conversational data access and the technical challenges involved in generating accurate database queries and interpreting user requests.

Despite these developments, the use of language models in operational decision-support systems requires careful consideration of factual accuracy. Generated responses may not always correspond to the current state of an organisation's underlying data. Retrieval-augmented generation provides a related approach in which externally retrieved information is used to support language-model outputs [14].

In the proposed Smart Inventory system, the Manager Assistant follows a hybrid design that separates factual information retrieval from natural-language explanation. Routine operational questions, including current stock availability, product information, and stock movement status, are handled through predefined application logic and database queries. The language model is used for open-ended explanations and interpretation of information supplied by the application.

This approach aims to reduce dependence on generated responses for operational facts while retaining the accessibility of a conversational interface. The Manager Assistant therefore acts as an additional decision-support mechanism alongside the forecasting, inventory intelligence, and stock movement modules, rather than independently executing inventory decisions.

2.4. Existing Systems, Products and Techniques

Existing approaches to retail inventory management can be classified into manual inventory management, rule-based enterprise resource planning (ERP) systems, forecasting and planning software, and research-based stock transshipment optimization methods.

Manual and spreadsheet-based approaches rely on inventory reports, historical sales information, and managerial judgement to identify shortages and excess stock. Although these approaches provide flexibility and are relatively straightforward to implement, the identification of suitable stock transfer opportunities depends heavily on the availability and interpretation of accurate inventory information.

ERP and inventory management platforms provide more structured mechanisms for monitoring stock levels, managing replenishment, and recording inventory movements. For example, Oracle NetSuite supports inventory transfer orders between locations, including approval, fulfilment, in-transit tracking, and receipt processes

[26]. These capabilities provide operational control and visibility over inventory movement.

Odoo Inventory provides another example of structured inventory management. Its replenishment functionality supports minimum and maximum stock thresholds, automatic or manual replenishment triggers, and forecasted inventory quantities [27]. Odoo also supports inter-warehouse replenishment through configured routes and stock transfer operations [28]. These capabilities demonstrate that established inventory platforms can provide rule-based replenishment and inter-location stock movement functionality.

A separate category consists of statistical and machine learning-based forecasting approaches. Classical techniques, including exponential smoothing and ARIMA, remain established forecasting methods and useful benchmarks [12]. More recent machine learning approaches, including gradient-boosted tree models, have demonstrated strong performance in hierarchical retail demand forecasting [2]. However, producing a demand forecast does not, by itself, establish how inventory should be redistributed between stores.

Research on lateral transshipment and multi-location inventory optimization addresses the redistribution of stock between locations [3], [6]. These approaches provide theoretical and mathematical foundations for determining stock movements under inventory and operational constraints.

The proposed Smart Inventory component combines several of these functions within a single application. It integrates XGBoost-based demand forecasting, inventory health assessment, deterministic source-store recommendations, natural-language explanations, and a controlled stock movement workflow with transaction logging.

The comparison presented in Table 2.1 is a synthesis of the reviewed literature and selected product documentation. It identifies the functions emphasised by each approach rather than claiming that commercial products lack features not examined in this review.

Table 2.1: Indicative comparison of approaches

Approach	Demand forecast	Branch-level transfer support	Explanation of decisions	Execution workflow
Manual and spreadsheet-based management	Manual estimates and historical trends	Ad hoc, based on managerial judgement	Informal, based on experience	Manual procedures
Rule-based ERP replenishment (e.g., NetSuite, Odoo) [26],[28]	Rule-based replenishment and stock projections; varies by product	Supported in documented transfer workflows	Configured rules and operational records; AI explanation not evaluated	Transfer and replenishment workflows
Forecasting research [1], [2], [15]	Statistical and machine learning forecasts	Not the primary focus of cited studies	Varies by study	Not evaluated
Transshipment optimization research [3], [6]	Uses demand estimates or assumptions	Optimization-based transfer decisions	Mathematical model and constraints	Not evaluated
Proposed Smart Inventory component	XGBoost-based daily, monthly and quarterly forecasts	Deterministic ranked recommendations	Language-model explanations based on supplied system facts	Approval, execution, inventory updates and audit logging

2.5. Gap Synthesis and Conceptual Framework

Three observations follow from the review. First, forecasting is well studied and tree ensembles are a reasonable choice for tabular retail data, but forecasts are seldom carried through to a specific stock-movement decision. Second, transshipment research provides useful models, but they are not usually packaged as interactive tools with explanation and workflow control. Third, explainability research has focused on model behaviour, and the use of language models to explain rule-based operational decisions is an open engineering question in which grounding and cost control matter.

These observations shaped the conceptual framework in Figure 2.1. Historical and contextual data feed a forecasting layer. The forecasts and current stock feed an inventory intelligence layer, which identifies shortages and surpluses. A deterministic recommendation engine turns these into ranked transfer options. An explanation layer describes the engine's results without changing them, and a workflow layer controls execution and records what happened. The manager stays in the loop throughout, approving or rejecting recommendations, and can query the system through a chatbot.

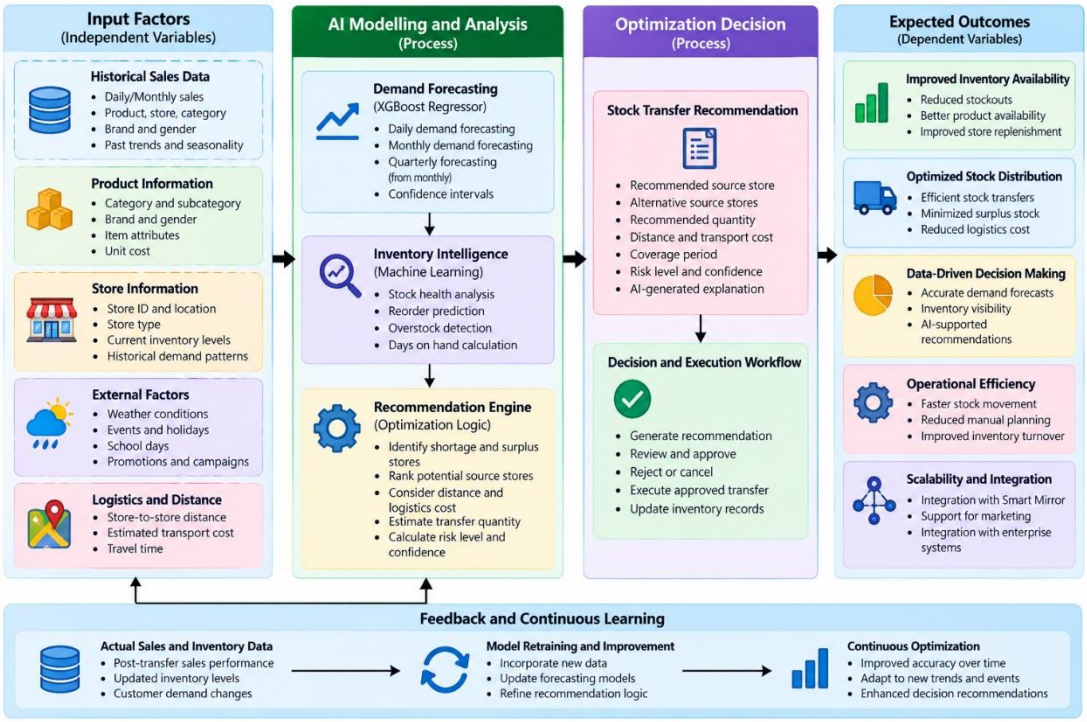


Figure 2.1: Conceptual framework for AI-driven inventory optimization

3. RESEARCH METHODOLOGY

3.1. Research Design

The study follows a design-and-evaluate approach. The component was built iteratively as a working product, and each stage was assessed with the type of evidence that suited it. The forecasting model was assessed quantitatively with error metrics on held-out data. The deterministic engine, the workflow and the chatbot were assessed through scenario-based functional testing. The explanation layer was assessed by checking that its output matched the facts supplied to it. Figure 3.1 summarizes the workflow.

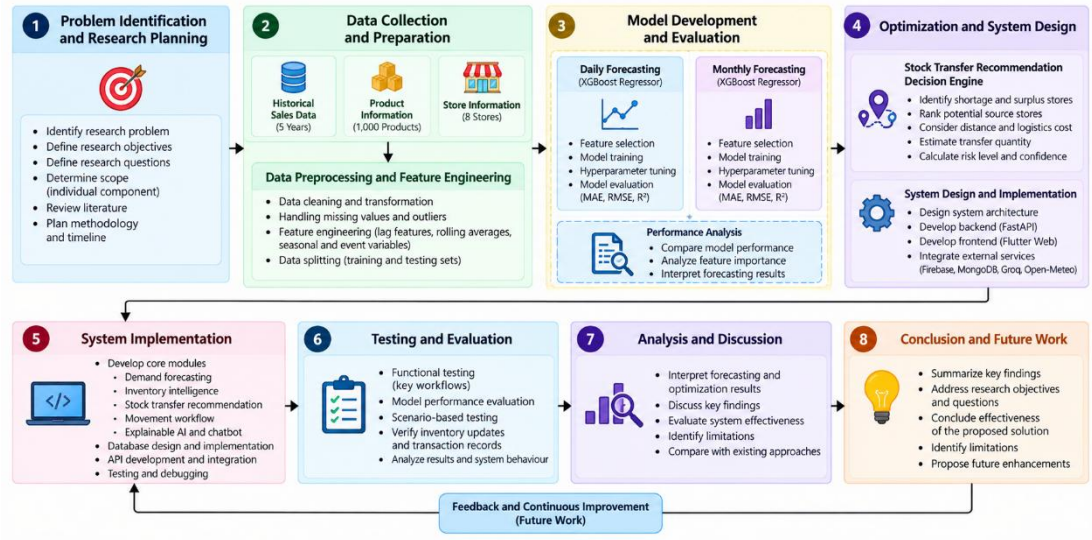


Figure 3.1: Research methodology workflow

The project also changed direction during its life, and that history matters for understanding the design. At the first progress presentation (PP1) the work was mainly a demand-forecasting prototype. The panel's feedback was positive; they described it as very good progress for the PP1 milestone. They also recommended improving the historical sales dataset by using approximately five years of data, because a longer history gives a better chance of identifying sales patterns. That recommendation directly shaped the second phase. After PP1, the historical coverage was extended to five years, the data was enhanced to represent seasonality and long-term trend, and additional influences (promotions, weather, weekends, holidays, festivals and school periods) were included. As the work progressed, it became clear that a forecast on its

own did not help a manager act. The scope therefore widened from forecasting to inventory intelligence, stock-transfer optimization, explainability and controlled execution, which is the component described in this thesis.

3.2. Data, Participants or Study Materials

3.2.1. Data source and its limits

This research used a fully synthetic retail dataset generated through Python scripts developed by the research team. Actual retail sales and customer transaction records were unavailable for research use because of commercial confidentiality. The synthetic dataset was not generated by extracting or reproducing patterns from confidential company sales data.

The dataset covers the period from 2021 to 2025 and represents eight retail stores and approximately 1,000 products. The final sales dataset contains 476,099 records, providing historical observations for the development and evaluation of the proposed forecasting and inventory intelligence models.

A complete daily dataset containing every product at every store throughout the five-year period would contain approximately 14.6 million product-store-day combinations. The available dataset is substantially smaller, representing approximately 3.3% of that theoretical combination count. Therefore, the dataset should not be interpreted as a complete daily inventory and sales history for every product at every store.

The exact record-generation and inclusion rules, including whether the generated observations represent selected selling days, active products, or sampled product-store combinations, require confirmation from the original dataset-generation scripts.

The synthetic dataset provides a controlled environment for developing and evaluating the proposed system. However, because the observations were artificially generated, forecasting results cannot be assumed to represent the performance achievable using real-world retail transactions.

The eight stores are listed in Table 3.1. Product brands in the catalogue include Emerald, Hustle, Miss Modano, Modano, King Street and Libera.

Table 3.1: Stores in the study

Store ID	Location	Role in the study
CP001	Nugegoda	Demand, stock and transfer node
CP002	Maharagama	Demand, stock and transfer node
CP003	Kandy	Demand, stock and transfer node
CP004	Kirulapane	Demand, stock and transfer node
CP005	Wattala	Demand, stock and transfer node
CP006	Colombo	Demand, stock and transfer node
CP007	Pelawatte	Demand, stock and transfer node
CP008	Malabe	Demand, stock and transfer node

3.2.2. Dataset attributes

Each sales record represents an observation associated with a particular store, product, and date. The dataset contains sales-related attributes and contextual information used to develop demand forecasting models. The principal attributes are summarised in Table 3.2.

Weather-related variables, including temperature, humidity, and rainfall, were incorporated as potential explanatory factors. Previous research has identified weather conditions, seasonality, promotional activities, and festivities as relevant considerations in fashion demand forecasting [29].

Calendar-related variables were also included to represent weekends, public holidays, festivals, and school periods. These features were intended to allow the forecasting models to learn possible relationships between calendar conditions and demand within the synthetic dataset. Their inclusion does not establish that each variable has a statistically significant effect on actual Sri Lankan retail sales.

Together, these attributes provide the historical, temporal, and contextual information required for the forecasting experiments described in the following sections.

Table 3.2: Attributes of the historical sales dataset

Attribute group	Fields
Identifiers	Store, product
Time	Date, month, day, day of week, weekend indicator
Sales	Sales quantity, selling price
Promotion	Promotion indicator and promotion percentage

Attribute group	Fields
Weather	Temperature, humidity, rainfall
Calendar events	Holiday, festival, school-period indicators
Derived history	Lag values and rolling statistics

3.2.3. Inventory data and the scalability decision

An early design considered storing inventory history as a time series, with a record for every product, store and day. Estimating that structure at the scale of this dataset gave more than two million inventory-history records. Carrying that much duplicated historical state inside the operational database would have complicated routine queries and made the data model harder to reason about, so the design moved to a compact current-state model instead.

The operational application database uses MongoDB through a compatibility layer that preserves several Firestore-style application interfaces following migration from Firebase Firestore. The migration was motivated by free-tier usage constraints encountered during development. The current-state inventory collection holds store-product stock records; stock movements and inventory transactions are stored separately. For approximately 1,000 products across eight stores, a fully populated store-product inventory view would contain about 8,000 records, but the actual collection size must be verified. The MongoDB adapter does not, by itself, establish that multi-document stock updates are atomic. [VERIFY: database counts, migration timeline and transactional safeguards.]

3.2.4. Participants

Stakeholder feedback was obtained from individuals working in retail and technical roles, including Inventory Managers, Merchandise Planners, IT Managers, Sales Assistants and Cashiers, as well as from university supervisors. Their practical comments informed iterative refinement of the prototype, particularly its stock visibility, movement workflows, operational relevance and usability. These were informal stakeholder consultations rather than a structured survey or controlled usability experiment. Participant counts, dates, individual comments and consent or feedback records will be documented when available; no quantitative satisfaction rate

or statistically representative conclusion is claimed. Functional tests, including source and destination stock updates and transaction logging, are reported separately in Chapter 5.

3.3. Experimental or Study Procedure

3.3.1. Data preparation and feature engineering

The research team developed Python scripts to generate synthetic retail observations covering the period from 2021 to 2025. The data were not derived from confidential retail sales records. The generated information included product, store, sales, weather, event, promotion, and inventory-related attributes.

The datasets were prepared for model development by aligning relevant observations using product, store, and date identifiers. Historical demand and contextual attributes were then used to construct the predictors required by the forecasting models.

Figure 3.2 illustrates the overall dataset preparation and feature-engineering process, beginning with synthetic data generation and continuing through data preparation, feature construction, chronological partitioning, and model-ready dataset creation.

The daily forecasting model uses the variables summarised in Table 3.3. Historical demand features include lag_1 and lag_7, intended to represent demand from the previous observation period and the corresponding period one week earlier. The rolling_mean_7 feature is intended to summarise recent demand. Additional predictors include pricing, promotional information, calendar indicators, and weather-related attributes.

Historical weather observations included in the training dataset are distinct from the weather information retrieved through the Open-Meteo API [18] during live forecasting requests, as described in Section 4.4.1.

To prevent target leakage, historical demand features should use only observations available before the prediction date. Rolling calculations must also preserve product-store boundaries so that observations from different time series are not combined incorrectly. The implementation of rolling_mean_7 and its treatment of series

boundaries require verification against the original feature-engineering notebook before the reported evaluation results can be considered fully validated.

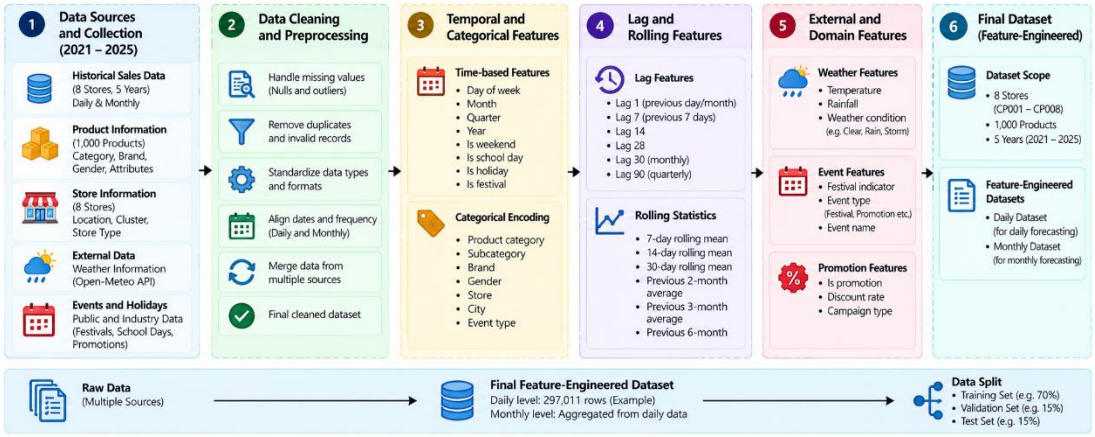


Figure 3.2: Five-year dataset preparation and feature-engineering pipeline

Table 3.3: Feature variables used by the daily forecasting model

Variable	Type	Purpose
lag_1	Numeric	Demand on the previous day
lag_7	Numeric	Demand seven days earlier, capturing weekly rhythm
rolling_mean_7	Numeric	Recent average demand level
Price	Numeric	Selling price effect
promotion percentage	Numeric	Size of promotional discount
month, day, day of week	Numeric or categorical	Seasonal and calendar position
weekend indicator	Binary	Weekend shopping effect
holiday indicator	Binary	Public holiday effect
festival indicator	Binary	Festival period effect
school-period indicator	Binary	School term or vacation effect
temperature, humidity, rainfall	Numeric	Weather effect

3.3.2. Model selection and training

XGBoost was selected as the primary forecasting algorithm based on its suitability for structured tabular data and its ability to model nonlinear relationships between explanatory variables. The dataset contains numerical and categorical-derived features representing historical demand, pricing, promotions, calendar conditions, and weather-related information. Gradient-boosted decision trees can model interactions among such features without requiring all relationships to be specified manually [4], [5].

The research developed separate daily and monthly forecasting models to support different planning horizons. The daily model was designed to predict short-term demand, while the monthly model provided forecasts at a broader planning level. These models form the forecasting foundation of the Smart Inventory component.

Table 3.4 presents a design-oriented comparison of the candidate forecasting techniques considered during model selection. The comparison describes their general methodological characteristics and suitability for the research requirements. Alternative algorithms were not trained and evaluated as part of the reported experiments; therefore, the comparison does not establish that XGBoost experimentally outperformed those alternatives.

Model performance was evaluated using the available synthetic dataset and the evaluation procedures described in this chapter. The reported results are presented in Chapter 6. Because the dataset is synthetic and the feature-engineering pipeline requires final validation, these results should be interpreted as prototype evaluation findings rather than evidence of performance on real retail transactions.

Table 3.4: Candidate forecasting techniques considered

Technique	Advantages for this project	Limitations and considerations
Moving average / seasonal naïve	Simple, transparent, computationally inexpensive; useful as a forecasting baseline	Basic versions rely on historical demand and do not directly incorporate price, promotion or weather variables
Exponential smoothing / ARIMA	Established time-series forecasting techniques; suitable for modelling trends and seasonality [12]	Conventional univariate implementations require separate modelling or configuration for individual series; external variables require extended formulations
Random Forest	Handles structured tabular data and nonlinear relationships; supports multiple explanatory variables	Requires historical feature engineering for time-series forecasting; performance relative to boosting must be established experimentally.
XGBoost (selected)	Suitable for tabular data; models nonlinear relationships and feature interactions; supports efficient training and prediction [4], [5]	Requires careful feature engineering, hyperparameter tuning and leakage prevention
Deep learning (LSTM, Transformer)	Can model sequential patterns and complex temporal relationships	May require additional training resources, data preparation and tuning; suitability depends on dataset characteristics

Two separate XGBoost regression models were trained for daily and monthly demand forecasting. The daily training notebook specifies 300 estimators, maximum tree depth 6, learning rate 0.05, subsample 0.8, column subsample 0.8 and random seed 42. The monthly notebook specifies 600 estimators, maximum depth 5, learning rate 0.03, subsample 0.9, column subsample 0.9 and random seed 42. The application provides seven-day daily forecasts and constructs quarterly forecasts from three monthly predictions; a separate quarterly model was not verified. These details distinguish model-training methodology from the runtime forecasting service.

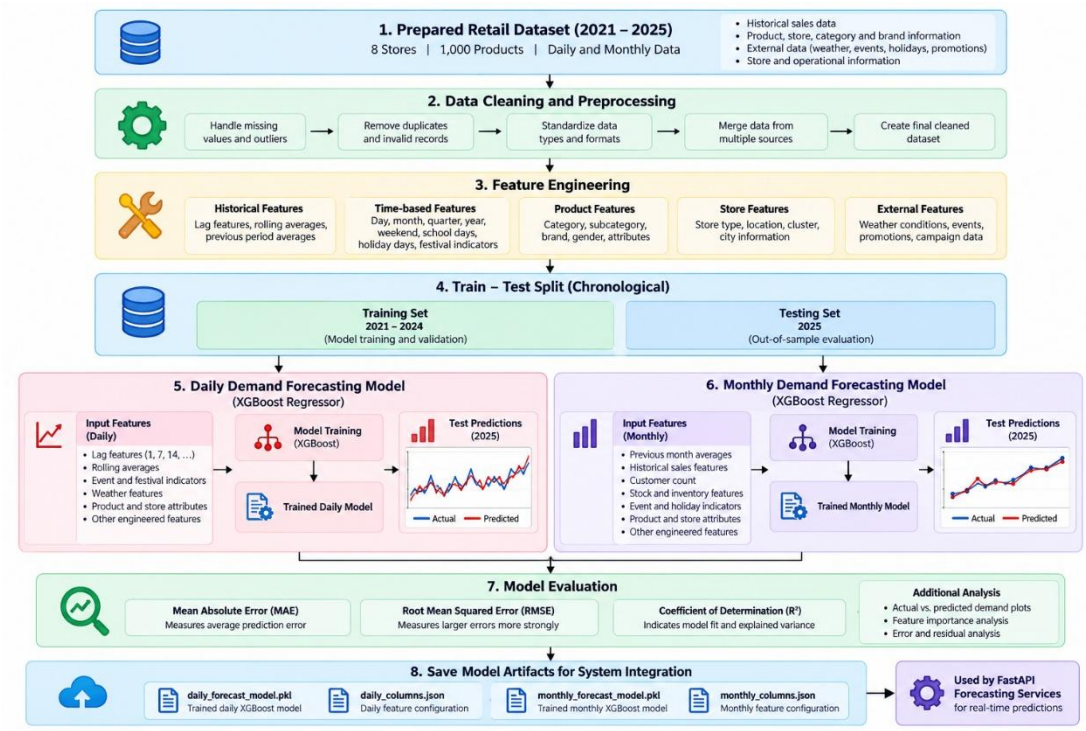


Figure 3.3: Machine-learning training and evaluation workflow

3.3.3. Development and verification procedure for the rest of the component

The remaining parts of the component were developed as backend services and verified by scenario testing. For the recommendation engine, I generated shortage situations from current inventory and forecast data, checked candidate source stores, and compared the ranking output with the criteria in Section 4.4. For the movement workflow I exercised each status transition, including invalid ones. For the explanation layer I compared explanation text to the structured facts it was given. For the chatbot

I asked a set of factual and open-ended questions and checked the answers against stored data. Section 5.4 lists these scenarios.

3.4. Measures and Evaluation Metrics

The performance of the developed demand forecasting models was evaluated using three regression evaluation metrics: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the Coefficient of Determination (R^2). These metrics were selected to assess forecasting accuracy, the magnitude of prediction errors, and the models' ability to capture variations in product demand. Table 3.5 summarises the evaluation metrics and their interpretations.

Mean Absolute Error (MAE) measures the average absolute difference between actual and predicted demand values. It provides an easily interpretable measure of forecasting error in the same units as product demand, with lower values indicating better predictive accuracy.

Root Mean Squared Error (RMSE) calculates the square root of the average squared prediction errors. Unlike MAE, RMSE places greater emphasis on larger forecasting errors, making it useful for identifying models that produce significant deviations from actual demand.

The Coefficient of Determination (R^2) measures the model's predictive performance relative to using the mean observed demand as a constant prediction. An R^2 value of 1 indicates perfect predictions, while a value of 0 indicates performance equivalent to the mean baseline. Negative values indicate performance worse than this baseline.

These three metrics were applied to evaluate the daily and monthly XGBoost forecasting models using the synthetic retail dataset. Their combined use provides a broader assessment of forecasting performance than relying on a single metric. The corresponding experimental results are presented and discussed in Chapter 6.

Table 3.5: Forecast evaluation metrics

Metric	Meaning	Reading
MAE	Average absolute difference between actual and predicted demand	Lower values indicate better accuracy; expressed in demand units

Metric	Meaning	Reading
RMSE	Square root of the average squared prediction error	Lower values indicate better accuracy; gives greater weight to large errors
R^2	Measures prediction performance relative to predicting the mean observed demand	Higher values are better; 1 indicates perfect predictions, 0 indicates performance equal to the mean baseline, and negative values indicate worse performance

Original Colab notebook outputs report daily MAE 0.5418, RMSE 0.9436 and R^2 0.9497, and monthly MAE approximately 14.2128, RMSE 26.3061 and R^2 0.9353. These are recorded experimental outputs, not independently reproduced measurements. A feature-engineering audit identified that rolling means were calculated after grouped shifts but without explicitly restricting the rolling window to each group. A corrected experiment and comparison against the original scores are required before treating the results as final. The application's displayed confidence percentages are heuristic decision-support indicators, not statistically calibrated prediction intervals.

3.5. Data Analysis

Both original training notebooks used a chronological holdout rather than a random train-test split. In the daily notebook, observations dated before 1 January 2025 form the training set and observations on or after that date form the test set. In the monthly notebook, years before 2025 form the training set and year 2025 forms the test set. This split evaluates the fitted models on a later synthetic period. However, grouped lag and rolling feature calculations must be audited for group-boundary contamination, and the feature construction order must be checked for leakage before the predictive performance can be interpreted confidently. The evaluation does not establish accuracy on real retail transactions.

3.6. Ethics, Privacy, Security and Responsible AI

This research used a fully synthetic dataset generated by the research team because actual retail sales and customer transaction records were confidential and unavailable for research purposes. The synthetic data were not constructed by extracting patterns from confidential company sales information. This approach reduced the need to process commercially sensitive transaction records during model development.

Nevertheless, the use of synthetic data introduces limitations. Artificially generated observations may not fully reproduce real customer behaviour, store-level demand variation, product life cycles, or operational constraints. Consequently, the reported forecasting performance cannot be assumed to represent accuracy under real retail conditions. These limitations are acknowledged when interpreting the evaluation results.

The Smart Inventory system uses Firebase Authentication as part of the group's shared sign-in infrastructure. Backend services were developed using FastAPI, with MongoDB Atlas used for data storage and Azure used to host the backend application. Inventory-changing operations are designed to pass through backend application services rather than relying solely on client-side processing.

Security considerations include protecting service credentials, restricting database access, validating authenticated requests, and enforcing appropriate permissions for stock movement approval and execution. The precise deployment configuration and enforcement of these controls require verification against the implemented backend and hosting environment.

Responsible AI considerations also influenced the system design. Stock transfer recommendations are generated through deterministic application logic, while the language model is used to explain information supplied by the system rather than independently determining transfer quantities or executing movements. This separation supports transparency and allows recommendations to be associated with identifiable inventory information and decision rules.

The explanation service is designed to use relevant product, store, demand, and inventory information as contextual input. The actual request payloads must be reviewed to confirm that unnecessary personal information is excluded from external model requests. Fallback responses are provided where the explanation service cannot return a usable result.

The stock movement workflow includes managerial approval and transaction logging to support human oversight and traceability. The system is therefore intended to assist inventory decision-making rather than replace managerial responsibility.

Generative AI tools were also used to assist with thesis preparation. Their use, the nature of the assistance received, and the author's responsibility for reviewing the resulting material are addressed in Appendix B.

3.7. Reproducibility and Research Artefacts

Research artefacts include the original daily and monthly Google Colab training notebooks, seven uploaded synthetic CSV datasets, saved model and feature definitions, FastAPI services, and the Flutter Web Inventory frontend under frontend-web. The individual researcher reports implementing all seven Inventory modules. MongoDB Atlas hosts the operational database, while the FastAPI backend is deployed on Microsoft Azure. Smart Mirror integration, including inventory availability sharing, was reported working and tested; Marketing integration currently supports one-way opportunity sharing, with bidirectional exchange reserved for future work. System test evidence and deployment screenshots will be added to the relevant chapters and appendices. [EVIDENCE REQUIRED: synthetic-data generation Python scripts; corrected rolling-feature experiment; final system test records.]

4. PRODUCT AND SOLUTION DESIGN

4.1. Requirements

The requirements were derived from the research problem, PP1 feedback, implementation constraints and iterative feedback from university and industry stakeholders. The individual component comprises seven Flutter Web modules: Dashboard, Forecasting, Inventory Intelligence, Optimization, Stock Movements, Analytics and Manager Assistant. Functional requirements cover multi-horizon demand forecasting, inventory health assessment, transfer source ranking, approval and execution of stock movements, decision explanations, operational analytics and access to inventory information through a conversational interface. Integration requirements include inventory availability sharing with Smart Mirror and information exchange with the Marketing component. [TO CONFIRM: the exact implemented Marketing return flow, API contract and corresponding test evidence.]

Table 4.1: Functional requirements

ID	Requirement	Objective
FR1	The system shall generate product-level demand forecasts for individual stores across supported daily, monthly and quarterly forecasting horizons.	O2
FR2	The system shall compare forecast demand with available inventory and classify store-product inventory conditions as SHORTAGE, HEALTHY or OVERSTOCK.	O3
FR3	The system shall calculate days of stock coverage, identify low-stock conditions and estimate when replenishment may be required.	O3
FR4	The system shall identify excess inventory and provide slow-moving or non-moving stock indicators where supported by available data.	O3
FR5	The system shall identify potential inter-store transfers by matching destination shortages with transferable surplus at other stores.	O4
FR6	The system shall rank eligible source stores and present alternative transfer candidates for managerial comparison.	O4
FR7	The system shall display the proposed transfer quantity, shortage coverage, source-to-destination distance, estimated logistics cost, operational risk and rule-derived recommendation confidence.	O4
FR8	The system shall generate natural-language explanations of inventory recommendations without allowing the language model to modify calculated decision values.	O5
FR9	The system shall support approval, rejection, cancellation and execution of stock movement recommendations while maintaining movement status information and history.	O6

ID	Requirement	Objective
FR10	Upon successful execution of an approved transfer, the system shall update the source and destination inventory balances and record the corresponding inventory movement information.	O6
FR11	The system shall validate stock movement status transitions and prevent repeated execution of the same movement.	O6
FR12	The system shall provide a conversational assistant capable of responding to supported questions about inventory, forecasts, recommendations, stock movements and analytics.	O7
FR13	The system shall provide product information, including available product image references and inventory availability, for integration with the Smart Mirror component.	O8
FR14	The system shall authenticate users through Firebase Authentication before allowing access to protected application functions.	O8

Table 4.2: Non-functional requirements

ID	Quality	Requirement
NFR1	Correctness	Inventory quantities, transfer costs, distances, and recommendation indicators shall be derived from stored data and application calculations rather than generated by the language model.
NFR2	Data integrity	Stock movement execution shall maintain consistent inventory balances and movement statuses, preventing the same approved transfer from being executed more than once.
NFR3	Explainability	The system shall provide understandable explanations for forecasting results, inventory assessments, and transfer recommendations, including comparisons with eligible alternative source stores where available.
NFR4	Availability of explanations	The system shall provide a deterministic fallback explanation when the external Groq API is unavailable, rate-limited, or returns an unusable response.
NFR5	Cost and quota efficiency	Previously generated explanations shall be reused through MongoDB caching when the underlying explanation context remains applicable, reducing unnecessary API requests.
NFR6	Scalability of storage	The MongoDB data structure and backend services shall support increasing volumes of products, stores, inventory records, and stock movement history without requiring fundamental architectural changes.
NFR7	Maintainability	The backend shall organise API routes, data schemas, and business logic into separate components to support independent maintenance, testing, and future enhancements.
NFR8	Usability	The Flutter Web interface shall present inventory indicators and recommendations in a clear format, allowing managers to review supporting information and available alternatives.
NFR9	Security	The system shall authenticate users through Firebase Authentication and process inventory-changing operations through controlled backend endpoints.

ID	Quality	Requirement
NFR10	Performance	The system shall provide responsive access to inventory information, forecasts, and recommendations under the intended operating conditions. Response times shall be evaluated through application testing where measurements are available.

4.2. Component Architecture Within the Complete Solution

The solution adopts a layered client-server architecture. The Flutter Web Inventory application communicates with a Python FastAPI backend, which orchestrates forecast inference, inventory calculations, deterministic transfer ranking, workflow operations and explanation services. MongoDB Atlas stores operational collections, while Firebase Authentication supports application sign-in. The backend is deployed to Microsoft Azure; this does not imply that the frontend or MongoDB is hosted on Azure. The system also exchanges inventory information with Smart Mirror and integrates with the group Marketing component. The current source evidence establishes an Inventory-to-Marketing opportunity submission path; the user reports that full bidirectional integration is implemented, but the reverse interface and its execution evidence remain to be verified before the thesis describes its detailed mechanism.

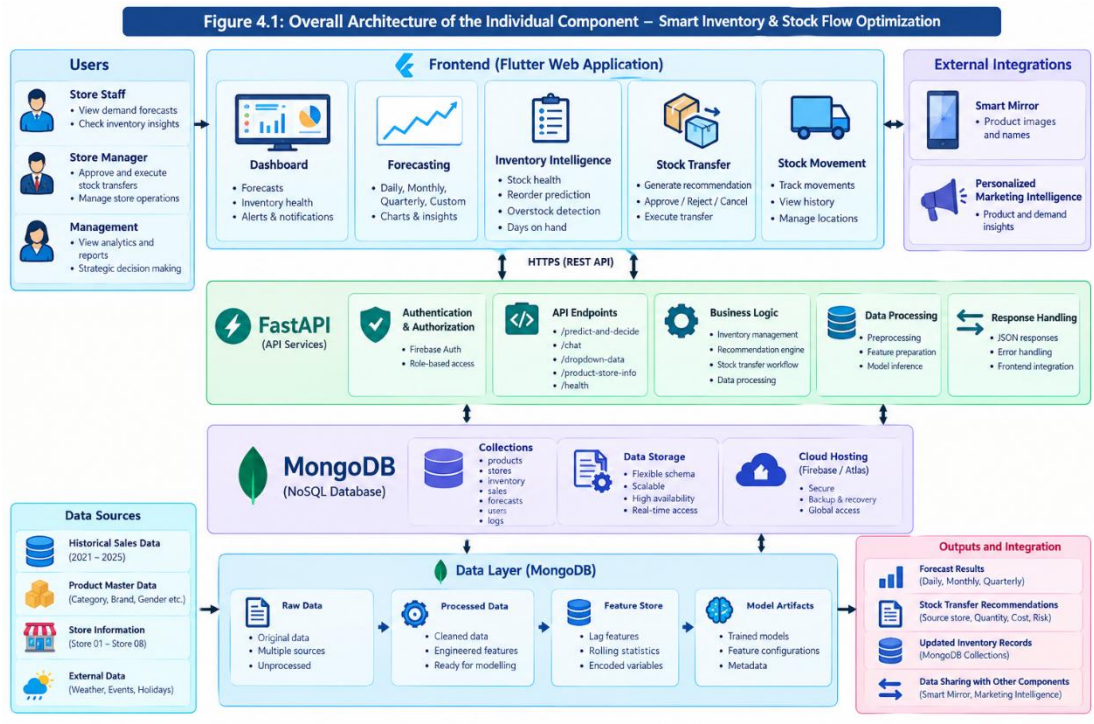


Figure 4.1: Overall architecture of the individual component

The backend separates API routes, domain services, data-access functions and model artifacts. Routes expose validated HTTP operations; services compute forecasts, inventory indicators, candidate rankings and stock-movement transitions; MongoDB Atlas stores operational records and workflow history. External integrations supply weather and route-distance information, and Groq is used to generate explanations from structured results. This division separates deterministic business decisions from language-model narration and supports traceability of inventory changes.

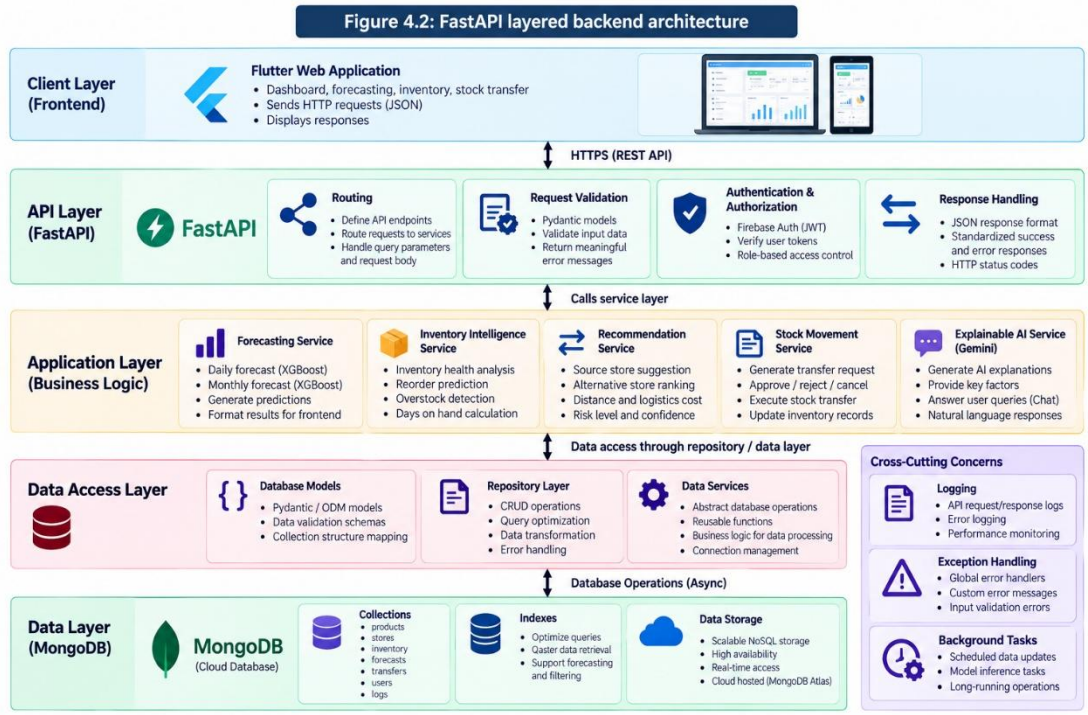


Figure 4.2: FastAPI layered backend architecture

4.3. Design Alternatives and Decisions

Design decisions were evaluated against explainability, development time, operational traceability and the constraints of a synthetic research dataset. XGBoost models are used for demand forecasting, whereas source-store selection uses explicit ranking rules rather than a learned policy. MongoDB Atlas stores current stock and workflow documents instead of a fully expanded daily stock-history collection. The Flutter Web application exposes seven individual Inventory modules; Azure hosts the FastAPI backend. The system also interfaces with Smart Mirror and Marketing as group-level integrations.

Table 4.3: Key design decisions and alternatives

Decision	Alternative considered	Reason for the choice
Deterministic recommendation engine	Let a language model choose the source store	Stock, cost and distance must be reproducible and auditable, and language models can produce ungrounded output [9]
Language model as explainer only	No explanation, or model-based decision making	Managers need reasons before they trust a recommendation, and grounding the model in supplied facts limits error [8], [14]
Cache explanations in MongoDB	Generate on every view	Reduces requests, latency, quota use and cost, and keeps repeated explanations consistent
Single Groq API for both explanations and the chatbot	Separate providers for each purpose	One inference provider to integrate and monitor for quota, while the two uses stay logically separate: grounded explanation text and open-ended chatbot answers
Live Open-Meteo weather at runtime	Rely only on historical calendar-based weather patterns	Lets the forecasting service reflect current conditions rather than only seasonal averages baked into the training data
Deterministic fallback explanations	Show an error when the model is unavailable	The manager can still see a readable reason when the external service fails
MongoDB holds current inventory state	Store full inventory history as time-series documents	More than two million inventory-history records were estimated for that design, which would have duplicated state unnecessarily and complicated operational queries
Analytics computed in backend services	Precompute and store every analytic	Avoids storing large derived history and keeps a single source of truth
Rank several sources, keep alternatives	Return a single best source	Managers can inspect trade-offs, and Miller's observation that people ask contrastive questions supports showing alternatives [13]
Chatbot answers factual questions locally	Send every question to a language model	Faster, cheaper and more reliable for questions with a definite stored answer
Representative product images on GitHub Pages	Store image files in MongoDB or Firebase Storage	Avoids storing binary data in the operational database and avoids Firebase Storage billing
XGBoost forecasting	Statistical or deep learning models	See Table 3.4

The alternative of solving transfers as a formal optimization problem, as in the transshipment literature [3], [6], was considered and set aside. A ranked, rule-based approach is easier for a manager to understand and for me to verify, and it fits the project's aim of decision support with a human approver. The cost is that the engine does not guarantee a globally optimal set of transfers across all products and stores. Section 7.5 comes back to this.

4.4. Detailed Design

4.4.1. Forecasting service

The forecasting service loads separately trained XGBoost artifacts for daily and monthly demand prediction. Daily forecasts use calendar, lag, rolling, promotion, event and weather-related inputs; the seven-day horizon updates relevant temporal features recursively. Monthly predictions use aggregated historical features and associated context. Quarterly output is derived from monthly forecasts rather than a separately trained quarterly model. Forecast confidence values exposed by the application are heuristic indicators and must not be interpreted as statistically calibrated prediction intervals. The original notebooks evaluated 2025 as a chronological holdout, as detailed in Chapter 3.

4.4.2. Inventory intelligence

The Inventory Intelligence module combines current store-product inventory with estimated future demand. Its calculations identify stock health, reorder needs, overstock situations and days on hand, providing a structured input for optimization. Operational stock quantities are read from MongoDB Atlas. These assessments are decision-support indicators, not automatic stock movement instructions; transfer execution remains a separate controlled workflow.

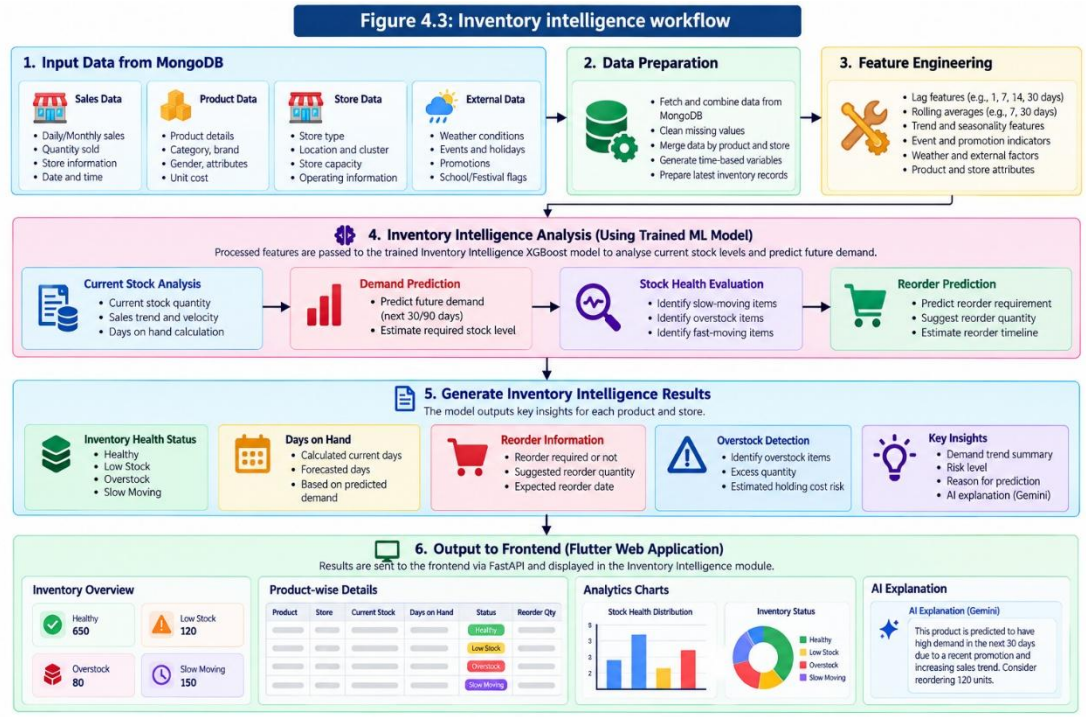


Figure 4.3: Inventory intelligence workflow

4.4.3. Transfer recommendation engine

The Transfer Recommendation Engine is a core component of the proposed Smart Inventory system. It identifies suitable source stores that can supply inventory to a destination store experiencing an actual or predicted stock shortage. The engine uses a deterministic, rule-based approach to evaluate inventory availability, transferable surplus, shortage coverage, reorder-level protection, transportation distance, estimated logistics cost, and operational risk.

The recommendation process begins when the Inventory Intelligence module identifies a stock shortage for a particular product at a destination store. The engine examines inventory information from other stores to identify potential sources holding the same product. Candidate stores are evaluated according to their available stock and the inventory quantities that should be retained to support their own expected demand.

An important consideration is preventing a transfer from creating a shortage at the supplying store. Therefore, the engine considers reorder-level protection and remaining source inventory when determining whether a store is suitable for

transferring stock. The proposed transfer quantity is evaluated against the destination shortage and the transferable inventory available at the source.

For each eligible source store, the engine calculates or retrieves the information required to evaluate the proposed transfer. This includes the transferable quantity, percentage of shortage coverage, remaining source inventory, distance between stores, and estimated logistics cost. The engine also produces an operational risk classification and a rule-derived recommendation confidence indicator.

Table 4.4: Recommendation criteria used by the deterministic engine

Criterion	Question it answers	Data source
Shortage at destination	How many units are required at the destination store?	Inventory intelligence output
Transferable surplus	How many units can the source store provide?	Current inventory, forecast information and engine calculations
Reorder-level protection	Would the transfer reduce source inventory below its required threshold?	Inventory collection and engine rules
Transfer quantity	How many units are proposed for transfer?	Recommendation engine calculation
Coverage percentage	What percentage of the destination shortage can be fulfilled?	Recommendation engine calculation
Remaining source inventory	How many units remain at the source after the proposed transfer?	Current inventory and transfer quantity
Distance	What is the distance between the source and destination stores?	Store-distances collection (Google Maps values where available)
Estimated logistics cost	What is the approximate transportation cost?	Distance-based cost estimation
Risk level	What operational risk is associated with the proposed transfer?	Deterministic engine classification
Recommendation confidence	How strongly do the engine rules support the recommendation?	Rule-derived confidence calculation

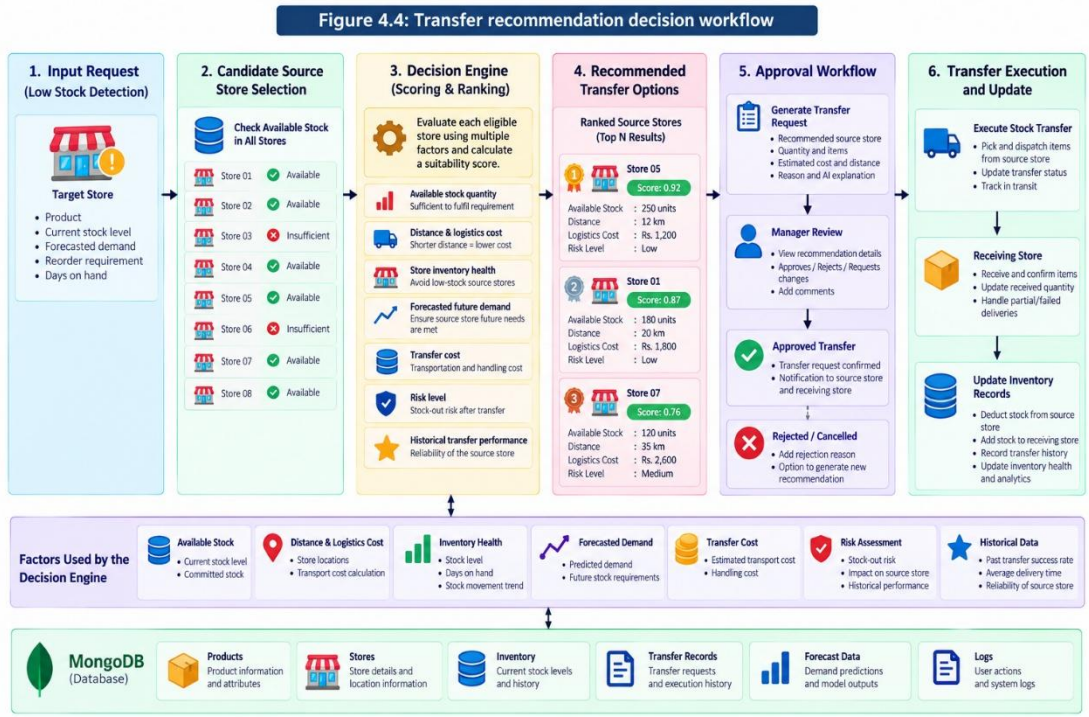


Figure 4.4: Transfer recommendation decision workflow

The engine ranks candidate source stores according to predefined decision criteria. Shortage coverage is prioritised, followed by logistics cost, distance, travel time, and remaining inventory buffer considerations. This approach is intended to balance the destination store's inventory requirements with the operational implications for the supplying store.

A source store with a large inventory surplus is not necessarily the most suitable option if the transfer involves substantially greater transportation costs or operational risks. Similarly, a nearby store may be considered a useful alternative when it can fulfil a significant proportion of the shortage while retaining sufficient inventory for its own operations.

The engine provides a recommended source store together with alternative candidates where eligible alternatives exist. Presenting multiple candidates allows managers to review the trade-offs between shortage coverage, transportation requirements, and remaining source inventory before approving a movement.

Store-to-store distance information is maintained separately in the MongoDB store-distances collection. Verified Google Maps distance values were used where available.

These distances support the evaluation of transportation requirements and estimated logistics costs. The precise calculation used for logistics cost, together with the handling of missing distance records, requires confirmation against the implemented recommendation service.

Operational risk and recommendation confidence are also derived from application-defined rules. The risk indicator is intended to communicate relevant operational concerns associated with a proposed transfer, while the confidence indicator provides an additional measure for comparing recommendations. The displayed confidence percentage is heuristic and should not be interpreted as a statistically calibrated probability of successful transfer.

Figure 4.4 illustrates the conceptual decision workflow. After a destination shortage is identified, the engine searches for potential source stores, evaluates their transferable inventory, applies stock protection considerations, and calculates the relevant recommendation criteria. Eligible candidates are then ranked, and the recommended source and alternatives are presented for managerial review.

The workflow also considers situations where no source can fully satisfy the shortage. In such circumstances, the available candidates must be assessed according to their achievable coverage. The precise implemented behaviour for partial coverage, equal-ranked candidates, and cases with no eligible source remains subject to verification against the backend recommendation logic.

The recommendation output supports the Stock Movement module, through which a proposed movement can enter the RECOMMENDED stage and proceed through the relevant approval and execution workflow. A recommendation does not itself represent a completed inventory transfer. Stock movements remain subject to managerial review and the application's movement-processing controls.

Unlike a machine learning-based source-selection model, the recommendation engine relies on deterministic calculations and decision rules. This separation allows demand forecasting to provide predictive information while the transfer recommendation process remains based on explicit operational criteria. The approach supports traceability, consistent decision-making, and managerial oversight.

Overall, the Transfer Recommendation Engine connects demand forecasting and inventory intelligence with actionable inter-store stock movement recommendations. By considering shortage coverage, source-store protection, logistics-related factors, and operational risk, the engine provides structured decision support for inventory redistribution across the retail network.

4.4.4. Explainable AI layer

The Explainable AI Layer was developed to improve the interpretability of forecasting results, inventory assessments, and stock transfer recommendations generated by the Smart Inventory system. Although the underlying forecasting models and recommendation engine produce quantitative results, these outputs may require additional interpretation before inventory managers can effectively use them for operational decision-making. Therefore, the explainability layer converts structured system information into natural-language explanations that are easier for managers to understand.

The system integrates the Groq API to generate natural-language explanations from information already calculated by the forecasting, inventory intelligence, and deterministic transfer recommendation modules. The language model functions exclusively as an explanation component and does not independently determine source stores, calculate transfer quantities, modify inventory balances, approve stock movements, or execute inventory transfers. These responsibilities remain within the application's deterministic decision logic and controlled stock movement workflow.

The explanation process begins when a forecasting result, inventory assessment, or transfer recommendation becomes available. The backend prepares a structured explanation context containing relevant factual information. For transfer recommendations, this context includes the destination store, identified shortage, recommended source store, proposed transfer quantity, shortage coverage percentage, alternative source stores, remaining inventory, transportation distance, estimated logistics cost, operational risk, and recommendation confidence.

The structured context is passed to the explanation service, which prepares a request for the Groq API. The language model then generates an explanation describing the

recommendation and the operational factors considered by the deterministic engine. This approach allows managers to understand why a particular recommendation was produced without requiring them to interpret the underlying calculations individually.

The explainability layer supports several types of explanation, including the reasoning behind a stock transfer recommendation, justification for selecting a particular source store, comparison of alternative source stores, and interpretation of trade-offs between inventory availability, shortage coverage, transportation distance, logistics cost, and operational risk. It also supports explanations of forecasting and inventory intelligence outputs, as well as summaries of completed stock movements and their operational implications.

To improve efficiency, the system incorporates an explanation caching mechanism using MongoDB. When an explanation is requested, the application checks whether a reusable explanation is available for the relevant recommendation or result. If a suitable cached explanation exists, it can be returned without generating a new response through the Groq API. Otherwise, the application requests a new explanation and stores the generated result for subsequent reuse.

Caching helps reduce repeated external API requests, improve response time, minimise service usage, and maintain consistency when managers review the same recommendation multiple times. However, cached explanations must remain consistent with the underlying operational information. Changes in inventory quantities, transfer recommendations, or stock movement status may affect the validity of an earlier explanation. The precise cache-key structure and cache invalidation conditions require confirmation from the implemented backend service.

The system also includes a fallback explanation mechanism to support continued operation when the Groq API is unavailable, rate-limited, or unable to provide a usable response. In such situations, the application can generate a local explanation using predefined rules and the structured factual context. This mechanism ensures that essential recommendation information remains accessible without depending entirely on the availability of the external language-model service.

A key architectural principle is the separation of explanation generation from operational decision-making. The forecasting models and recommendation engine complete their calculations before the explanation layer is invoked. The language model receives these calculated results as input and has no authority to modify the original decisions or initiate stock movements. Consequently, the recommended source store, proposed transfer quantity, risk classification, and other operational indicators remain determined by the underlying application logic.

This separation improves the traceability of recommendations and reduces the risk of allowing generated language to control inventory operations. Nevertheless, natural-language explanations may contain inaccuracies or interpretations that are not fully supported by the supplied context. Therefore, AI-generated explanations are treated as supplementary decision-support information, while the structured outputs of the forecasting, inventory intelligence, and recommendation modules remain the authoritative source for operational decisions.

Figure 4.5 illustrates the architecture of the Explainable AI Layer, including the preparation of structured factual information, MongoDB cache lookup, Groq API integration, local fallback generation, and presentation of explanations to inventory managers.

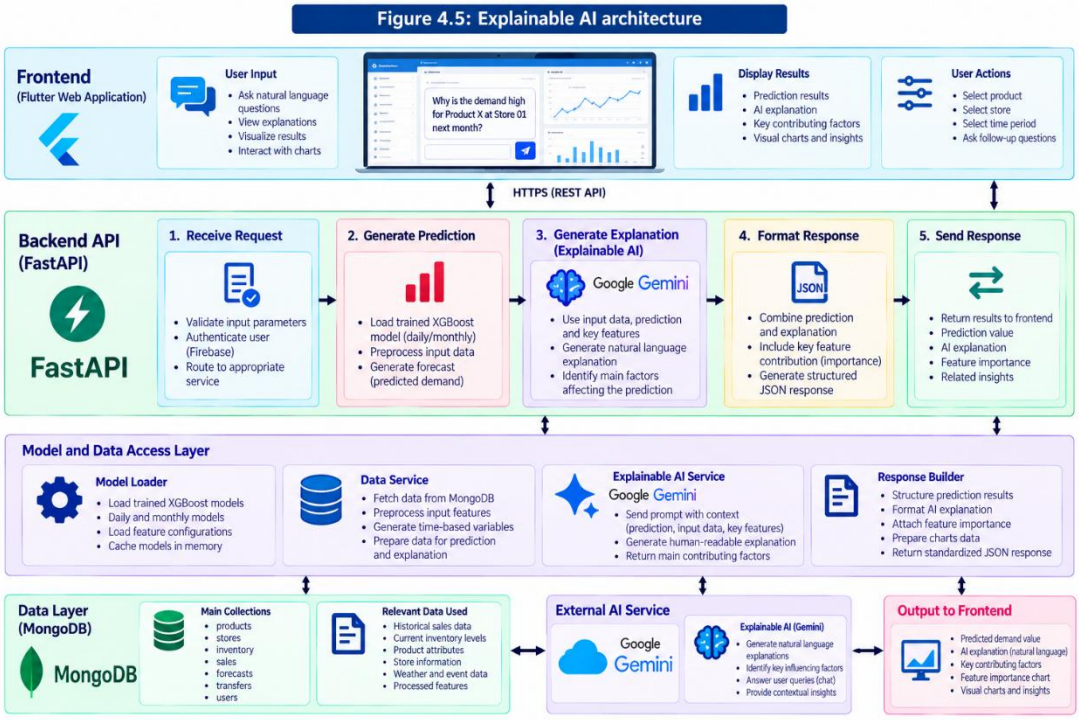


Figure 4.5: Explainable AI architecture

Overall, the Explainable AI Layer enhances the usability of the Smart Inventory system by translating technical results into understandable business explanations. The integration of deterministic decision logic, Groq-based natural-language generation, MongoDB caching, and fallback handling provides a structured approach to explainable decision support while preserving managerial control over inventory-related operations.

4.4.5. Manager chatbot

The Manager Assistant provides a conversational interface for retrieving and interpreting available Inventory information. It routes recognized user requests to the relevant data and service operations and uses an explanation layer to present understandable responses. Conversation history is managed separately from stock movement records. The assistant is a query and interpretation interface, not an autonomous actor that approves or executes transfers.

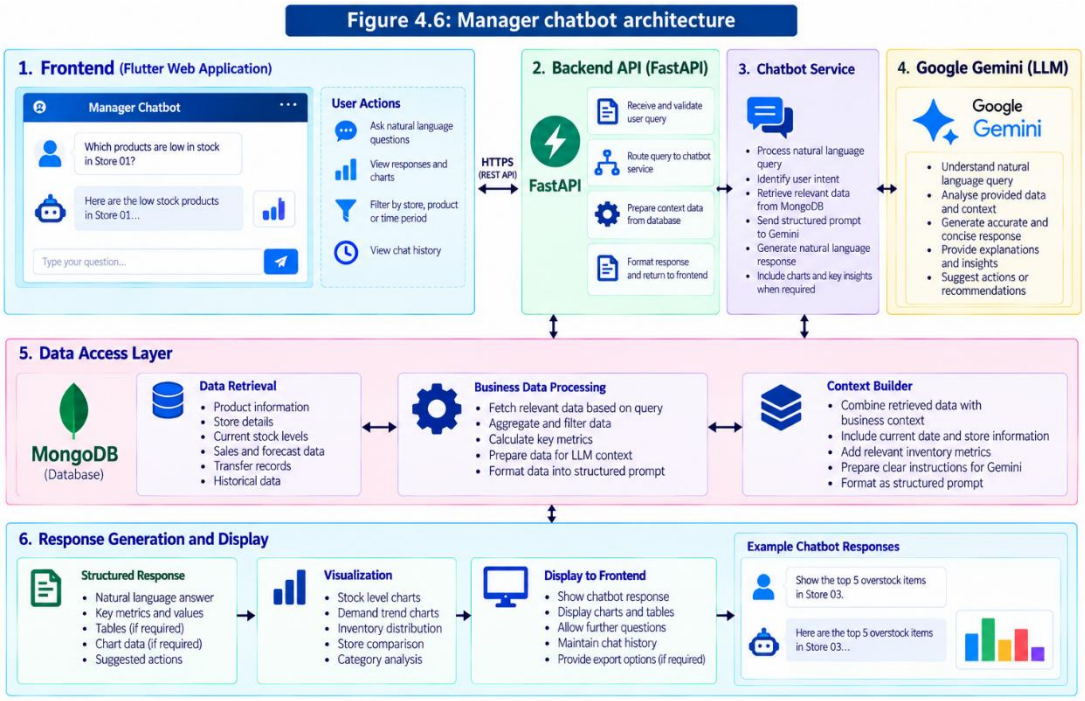


Figure 4.6: Manager chatbot architecture

4.4.6. Stock movement workflow

The Stock Movements module uses a controlled lifecycle with the statuses RECOMMENDED, APPROVED, IN_PROGRESS, EXECUTED, REJECTED and CANCELLED. Approval and execution are distinct operations. An executed movement updates source and destination stock records and creates inventory transaction evidence and status history. The application includes guards against duplicate execution; however, the existence of a multi-document database transaction or guaranteed atomic rollback has not been established and is not claimed here.

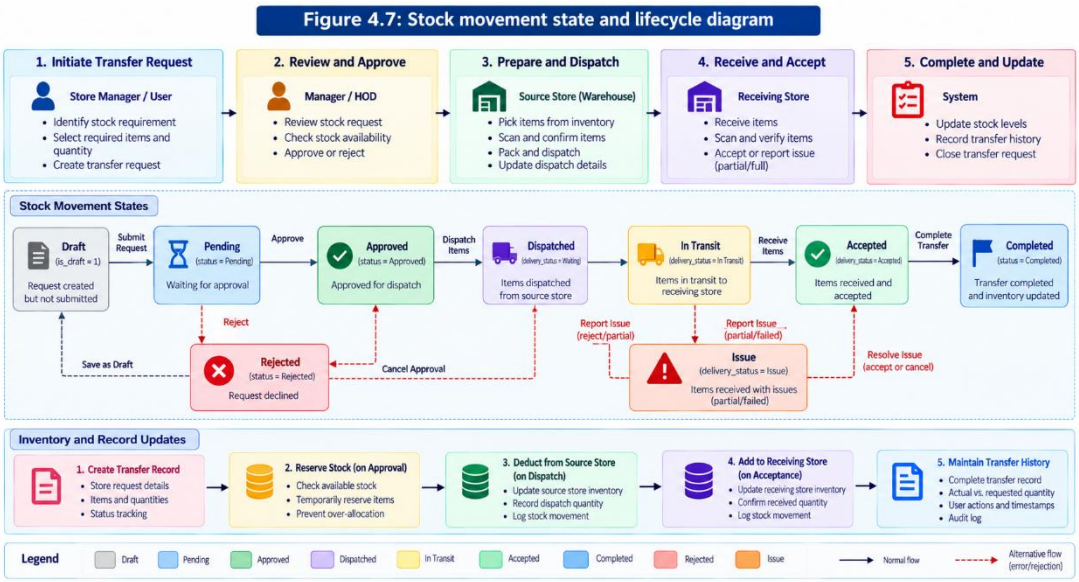


Figure 4.7: Stock movement state and lifecycle diagram

Table 4.5: Stock movement statuses

Status	Meaning	Possible next status
RECOMMENDED	Created by the engine, waiting for a decision	APPROVED, REJECTED, CANCELLED
APPROVED	Approved by a manager	IN_PROGRESS, CANCELLED
IN_PROGRESS	Transfer under way	EXECUTED
EXECUTED	Stock has moved and inventory is updated	None
REJECTED	Manager declined the recommendation	None
CANCELLED	Withdrawn before execution	None

For an executed transfer, the system reduces the source-store quantity and increases the destination-store quantity, records before-and-after values, creates transaction and workflow-history entries, and marks the movement EXECUTED. The researcher reports that this end-to-end behavior was tested successfully. Chapter 5 will identify the precise test scenario, expected outcome and observed evidence once the test records are supplied.

4.4.7. Data model

The operational data model is implemented in MongoDB Atlas. Key collections include products, stores, inventory_current, stock_movements, inventory_transactions, optimization_candidates, optimization_history, decision_workflows and marketing_opportunities. Forecast artifacts are maintained

separately from operational documents. The schema supports inventory state, recommendation alternatives, approval history, transfer execution records and cross-component opportunity exchange. Collection names and relationships should be shown in Figure 4.8 without implying relational foreign-key enforcement.

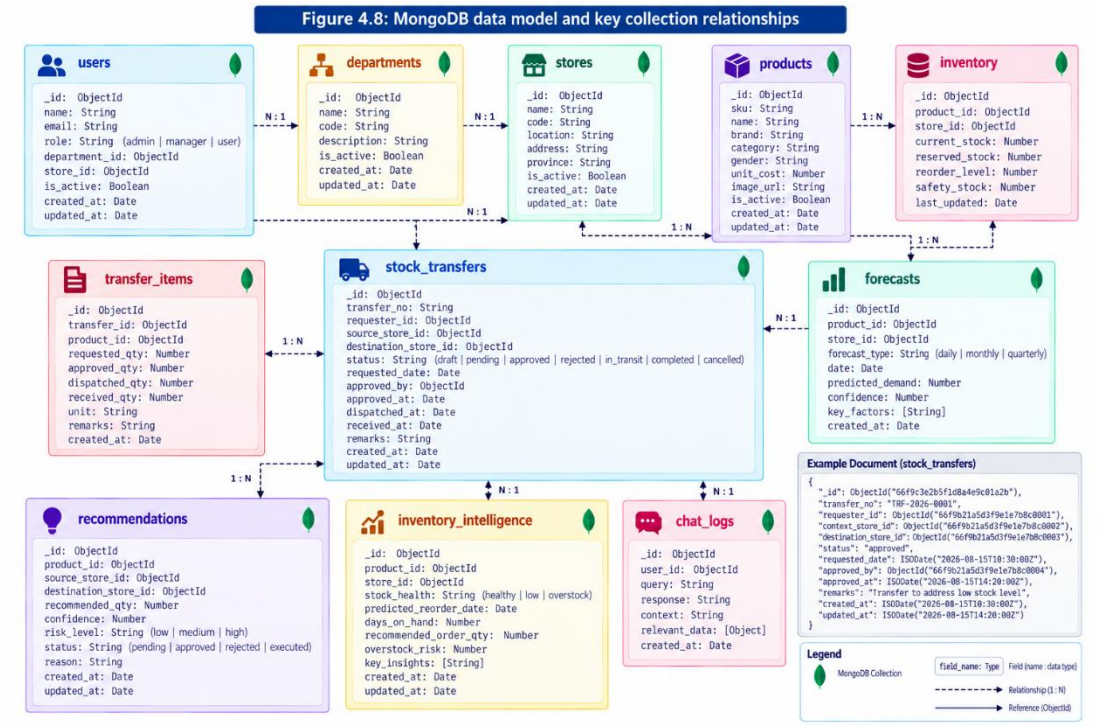


Figure 4.8: MongoDB data model and key collection relationships

Table 4.6: Key MongoDB collections

Collection	Purpose
alert_items	Stores inventory-related alerts and notifications, including stock shortage and other inventory conditions.
analytics_summary	Maintains summary information used by the analytics dashboard.
brand_analytics	Stores aggregated analytics information grouped by product brand.
category_analytics	Stores aggregated analytics information grouped by product category.
decision_workflows	Maintains information related to decision-making workflows and their processing states.
gender_analytics	Stores aggregated analytics information based on product gender classifications.
inventory_current	Maintains current inventory information for store-product combinations.
inventory_transactions	Records inventory transactions and stock quantity changes for tracking and auditing.

Collection	Purpose
manager_chat_sessions	Stores conversational session information associated with the Manager Assistant.
marketing_opportunities	Maintains information related to identified marketing opportunities and cross-module integration.
monthly_forecast_profiles	Stores monthly forecasting-related profiles used to support demand prediction and analysis.
optimization_candidates	Maintains candidate inventory transfer recommendations and associated evaluation information.
product_images	Stores product image references and related metadata for product visualisation and integration.
products	Maintains product master information, including product identifiers, descriptions, categories, and brands.
showroom_analytics	Stores analytics information associated with individual showrooms or retail stores.
stock_movements	Maintains inter-store stock movement records and their processing statuses.
store_distances	Stores distance information between retail stores to support transfer recommendations and logistics evaluation.
stores	Maintains master information for the eight retail stores.

4.5. Standards and Professional Practice

Professional practices include separation of API and business logic, Git-based version control, server-side control of stock-changing operations, explicit workflow states, and protection of API keys and database connection secrets. FastAPI provides OpenAPI-compatible endpoint documentation. Authentication is handled through Firebase Authentication, while MongoDB Atlas stores operational data. The design does not claim production-grade transactional guarantees or formal penetration-test results without corresponding evidence. Screenshots and final endpoint-level integration tests will be attached during thesis finalization.

The Smart Mirror integration shares product-related information and inventory availability; the researcher reports that it has been tested. Inventory-to-Marketing promotion opportunity sharing has been implemented. The broader bidirectional Marketing integration is part of the target design: Marketing can request an inventory feasibility assessment for a proposed campaign, and Inventory is intended to return a stock-based feasibility response. This reverse request-and-response workflow is partially implemented and still requires development and end-to-end testing.

Consequently, it must not be presented as a completed integration or as an evaluated outcome.

4.6. Cross-Component Integration Boundaries

The Marketing feasibility workflow begins when a Marketing user plans a campaign and requests a stock check for the relevant products, stores, quantities and campaign period. The Inventory component should assess current availability and relevant stock commitments, then return a supported/not-supported assessment with explanatory inventory information. The precise request schema, decision thresholds, authentication, error handling and response fields must be confirmed during implementation. A feasibility response is operational advice, not a marketing approval decision. The currently implemented Inventory-to-Marketing opportunity submission remains a separate, working direction of exchange.

5. IMPLEMENTATION, INTEGRATION AND TESTING

5.1. Development Environment and Technology Stack

The AI-Driven Smart Inventory and Stock Flow Optimization component was developed using a combination of modern web development frameworks, machine learning technologies, cloud-based services, and external application programming interfaces (APIs). The selected technology stack supports demand forecasting, inventory intelligence, inter-store stock transfer optimization, stock movement management, business analytics, and AI-assisted managerial decision-making. The development architecture separates the user interface, backend services, database operations, machine learning models, and external integrations to support modularity and maintainability.

The frontend application was developed using **Flutter Web and the Dart programming language**. Flutter Web provides a browser-based management interface through which users can access the dashboard, demand forecasting, inventory intelligence, optimization recommendations, stock movement workflows, analytics, and Manager Assistant. Its component-based architecture enables the reuse of interface elements across multiple modules, providing a consistent user experience.

The backend was implemented using **Python and the FastAPI framework**. FastAPI was selected because of its compatibility with Python-based machine learning libraries and its support for RESTful API development, request validation, and automatic API documentation. The backend coordinates communication between the frontend, forecasting models, inventory intelligence services, optimization engine, MongoDB database, and external APIs. Business rules for inventory assessment, transfer recommendations, approval workflows, and stock movement execution are implemented within the backend rather than delegated to the language model.

MongoDB Atlas was used as the managed cloud database for storing the operational information required by the component. The database contains collections for product and store master data, current inventory, forecasting-related profiles, optimization candidates, stock movements, inventory transactions, analytics, alerts, and Manager Assistant conversations. Its document-oriented structure supports the different data

representations required by the system. Firebase Authentication was integrated to provide user sign-in functionality without requiring a separately developed authentication service.

The demand forecasting functionality was developed using the **XGBoost machine learning algorithm**. Separate daily and monthly forecasting models support demand prediction at different time horizons. The monthly forecasting service loads a trained model and its associated feature-column configuration, prepares the input data, and generates non-negative demand predictions. Quarterly demand forecasting is implemented through three sequential monthly predictions rather than through a separately trained quarterly model. During this process, the service updates relevant historical demand features between successive predictions and calculates total quarterly demand by aggregating the three monthly outputs.

Weather-related information is incorporated through the **Open-Meteo API [18]**. The weather service supports short-range weather forecasts of up to 14 days and historical seasonal weather profiles based on the years 2021–2025. Weather requests use the geographical coordinates associated with individual stores. The retrieved information includes temperature, humidity, rainfall, precipitation, and weather conditions. For monthly and quarterly forecasting, the system generates historical seasonal weather profiles corresponding to the relevant calendar months. This approach provides seasonal weather inputs without relying on short-range forecasts for distant future periods.

The weather service also implements temporary in-memory caching to reduce repeated external API requests. Short-range weather responses are cached for 30 minutes, while historical seasonal weather profiles are cached for 24 hours. These caches are maintained within the Python application process and can be cleared through the weather service.

The **Explainable AI layer** was integrated using the Groq API [17]. The backend defines a configurable sequence of language models, with qwen/qwen3.6-27b as the primary model, followed by openai/gpt-oss-20b and openai/gpt-oss-120b as fallback models. The model identifiers are configured through environment variables, allowing

the model selection settings to be changed without modifying the application source code. Groq-based language generation supports natural-language explanations of forecasting results, inventory assessments, and stock transfer recommendations.

The Manager Assistant combines deterministic application logic with language-model-based responses to support inventory-related questions and managerial analysis. The language model is used for explanation and conversational interaction, while inventory calculations, transfer source selection, approval decisions, and stock movement execution remain controlled by the application's backend logic. This separation helps maintain consistency between operational decisions and the explanations presented to managers.

Additional technologies were integrated to support system functionality and development activities. Product images are referenced through stored image metadata, with representative images hosted using GitHub Pages. **Git and GitHub** were used for source code version control and development collaboration. Visual Studio Code and the relevant Python, Flutter, and Dart development tools were used during application implementation and testing. FastAPI's OpenAPI-based documentation facilities supported the inspection and testing of backend endpoints.

The backend application was deployed using **Microsoft Azure**, while MongoDB Atlas provides managed database hosting. This cloud-based arrangement separates application execution from database storage and supports access to backend services through the deployed API.

Table 5.1 summarises the technologies used in the implementation of the Smart Inventory and Stock Flow Optimization component.

Table 5.1: Technology stack

Layer	Technology	Use in the component
Frontend	Flutter Web (Dart)	Browser-based dashboard, demand forecasting, inventory intelligence, transfer optimization, stock movement management, analytics, and Manager Assistant interfaces.

Layer	Technology	Use in the component
Backend	FastAPI, Python	**RESTful APIs, request validation, business logic, machine learning model integration, and inventory operations.**
Database	MongoDB	Managed cloud storage for products, stores, inventory, forecast profiles, optimization candidates, stock movements, transactions, analytics, alerts, and conversational data.
Authentication	Firebase Authentication	User sign-in and authentication.
Machine learning	XGBoost	Daily and monthly demand prediction, with quarterly forecasts generated using three sequential monthly predictions.
Explainable AI	Groq API; qwen/qwen3.6-27b, openai/gpt-oss-20b, openai/gpt-oss-120b	Configured language models supporting natural-language explanations of forecasting, inventory intelligence, and optimization results.
Manager chatbot	Deterministic handlers and Groq API	Data-grounded responses to supported managerial queries and conversational explanations.
Weather Integration	Open-Meteo Forecast and Historical Weather APIs	Store-specific short-range weather forecasts and historical seasonal weather information for forecasting services.
Weather Caching	Python in-memory dictionaries	Caching short-range weather responses for 30 minutes and historical seasonal profiles for 24 hours.
Product Images	GitHub Pages	Hosting representative product images referenced by the application.
Backend Hosting	Microsoft Azure	Cloud deployment of the FastAPI backend application.
Source Control	Git and GitHub	Source code version management, collaboration, and change tracking.
Development Tools	Visual Studio Code, Python tooling, Flutter SDK, Dart SDK, FastAPI OpenAPI documentation	Application development, debugging, testing, and API inspection.

The selected development environment enabled the integration of machine learning functionality with operational inventory management through a unified web application. Flutter Web provides the presentation layer, FastAPI manages application

services and business rules, and MongoDB Atlas supports persistent data storage. XGBoost provides demand forecasting capabilities, while Open-Meteo supplies weather-related inputs and the Groq API supports natural-language explanations and conversational assistance.

Overall, the technology stack was selected to support the implementation of a modular, cloud-deployed inventory management component that combines predictive analytics, deterministic stock optimization, operational workflows, and explainable managerial decision support.

5.2. Component Implementation

The AI-Driven Smart Inventory and Stock Flow Optimization component was implemented as a set of interconnected backend services integrated with a Flutter Web application. The implementation covers data preparation and demand forecasting, inventory intelligence, stock transfer recommendations, AI-assisted explanation generation, and stock movement management. The FastAPI backend coordinates these services and provides the application programming interfaces through which the frontend accesses their functionality.

5.2.1. Data and forecasting

The synthetic retail dataset was prepared using Python-based data generation and preprocessing scripts following the data preparation pipeline presented in Figure 3.2. The preparation process included data cleaning, feature engineering, and the organisation of historical sales, product, store, calendar, and weather-related information into suitable inputs for machine learning.

Demand forecasting was implemented using XGBoost regression models trained separately for daily and monthly demand prediction. The models were trained offline and saved as serialized model files, together with the corresponding feature configurations. During prediction, the backend loads the required model and prepares the input features according to the structure used during training.

The forecasting functionality supports daily, monthly, and quarterly demand analysis. Monthly forecasting is performed using the trained monthly XGBoost model, with

historical seasonal weather features incorporated into the prediction input. These features are obtained through the Open-Meteo Historical Weather API and represent historical weather patterns for the relevant store and calendar month.

Quarterly forecasting is implemented through three sequential monthly predictions rather than a separately trained quarterly model. After each monthly prediction, the forecasting service updates relevant historical demand features before generating the prediction for the following month. The three predicted monthly demand values are then aggregated to obtain the quarterly demand estimate.

The forecasting service also produces a rule-derived confidence percentage based on the difference between predicted demand and a historical demand baseline. This value is intended as an indicative measure for managerial interpretation rather than a statistically calibrated prediction interval.

The performance of the trained daily and monthly forecasting models is evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2). The evaluation results are presented in Section 6.1.

5.2.2. Inventory intelligence and recommendation

The Inventory Intelligence service processes current stock information and demand forecasts to evaluate inventory conditions across products and retail stores. The service supports the identification of potential shortages, healthy inventory conditions, and overstock situations. It also provides inventory indicators such as estimated days of stock coverage and replenishment-related information, as described in Section 4.4.2.

The Transfer Recommendation Engine uses the inventory assessment results to identify potential opportunities for inter-store stock redistribution. When a shortage is identified at a destination store, the optimization service evaluates alternative source stores using the available inventory and transfer-related information.

The recommendation process considers relevant factors such as transferable stock quantity, destination demand coverage, remaining source inventory, store distance, estimated logistics cost, transfer risk, and rule-derived confidence. These factors support the comparison and ranking of alternative transfer options.

Candidate recommendations are maintained in the optimization_candidates MongoDB collection, allowing recommendation information to be accessed by the application. The frontend presents the recommended transfer and its available alternatives to support managerial review.

The core recommendation calculations are implemented through deterministic Python business logic rather than being generated by a language model. This separation ensures that the recommendation process is based on defined inventory and optimization rules. It also supports traceability, since the numerical factors contributing to a recommendation can be examined independently of the natural-language explanation.

5.2.3. Explanation generation and caching

The Explainable AI layer was implemented to convert forecasting, inventory intelligence, and transfer recommendation results into understandable natural-language explanations for managers.

The explanation process uses structured information produced by the forecasting and optimization services. This information provides the factual context for generating explanations of demand estimates, stock conditions, recommended transfers, and relevant trade-offs between alternative source stores.

The Groq API [17] was integrated to support language-model-based explanation generation. The backend defines qwen/qwen3.6-27b as the default primary model, with openai/gpt-oss-20b and openai/gpt-oss-120b configured as alternative models. These identifiers can be overridden through environment variables.

The explanation service is designed to keep natural-language generation separate from operational decision-making. The language model interprets supplied results for managerial presentation but does not determine transfer quantities, select source stores, modify inventory records, or approve stock movements.

The implementation also incorporates explanation reuse and fallback handling to reduce dependence on repeated external language-model requests. Reusing previously

generated explanations can reduce unnecessary API usage when the underlying recommendation information remains unchanged. A deterministic explanation based on the available recommendation data can provide a fallback when language-model generation is unavailable.

This approach supports the presentation of recommendation results while keeping inventory decisions under the control of the backend's deterministic business rules.

5.2.4. Movement workflow and inventory synchronization

The Stock Movement service implements the operational workflow described in Figure 4.7. It manages the progression of stock transfer requests through the supported movement statuses: RECOMMENDED, APPROVED, IN_PROGRESS, EXECUTED, REJECTED, and CANCELLED.

The workflow is designed to separate the recommendation process from the authorization and execution of stock transfers. A generated recommendation does not immediately change inventory quantities. Instead, the movement must progress through the applicable managerial approval and execution stages.

The backend validates movement status changes against the permitted workflow rules. This prevents unsupported transitions and helps maintain a consistent movement history.

During stock movement execution, the system processes the transfer between the selected source and destination stores. The execution logic is responsible for updating the corresponding inventory quantities and maintaining records of the stock movement. The stock_movements collection maintains movement information, while the inventory_current and inventory_transactions collections support inventory balances and transaction tracking.

Inventory synchronization is an important aspect of this workflow because the stock deduction at the source and stock addition at the destination must represent the same transfer operation. The implementation therefore requires safeguards against repeated execution and inconsistent inventory updates.

The completed movement is represented by the EXECUTED status, allowing managers to distinguish completed transfers from recommendations and movements awaiting further action.

Figure 5.1 illustrates the overall sequence from the generation of a stock transfer recommendation to the execution and recording of the corresponding inventory movement.

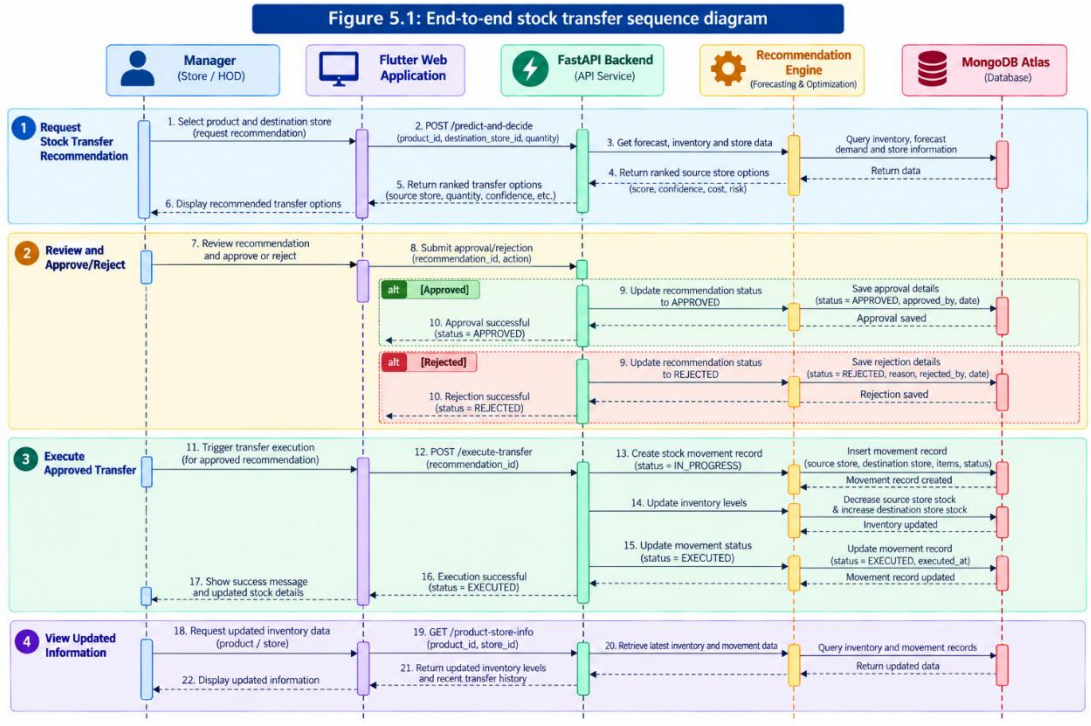


Figure 5.1: End-to-end stock transfer sequence diagram

5.2.5. Manager chatbot

The Manager Assistant was implemented as a conversational interface to support inventory-related information retrieval and managerial decision-making. It allows managers to submit natural-language questions and obtain responses concerning inventory availability, demand forecasts, stock transfer recommendations, movement statuses, pending approvals, and business analytics.

The chatbot service is integrated with the FastAPI backend and uses operational information maintained in MongoDB. Its architecture combines application-level

query handling with language-model-based response generation. For supported inventory-related questions, the backend retrieves relevant information from the system's operational data and uses it to construct responses. This approach enables managers to access inventory information without manually navigating multiple application modules.

The Groq API [17] provides natural-language generation capabilities for conversational questions requiring explanations or interpretation of available information. The backend defines a configurable language-model sequence consisting of qwen/qwen3.6-27b, openai/gpt-oss-20b, and openai/gpt-oss-120b. The use of operational data as context helps align generated responses with the information available in the application.

The Manager Assistant is designed as an information retrieval and decision-support interface rather than an autonomous inventory management agent. Its conversational responses do not independently approve stock transfers, change inventory quantities, or execute movement operations. These actions remain under the control of the relevant backend services and movement workflows.

The implementation provides an additional method for accessing forecasting, inventory, optimization, and analytics information through a unified conversational interface.

5.2.6. Backend module summary

The Smart Inventory backend was implemented using FastAPI and organised into separate API router modules according to their functional responsibilities. The application entry point, main.py, initializes the FastAPI application, configures Cross-Origin Resource Sharing (CORS), registers the API routers, and initializes Firebase during application startup.

The backend application is identified as **Smart Inventory AI Service**, version **2.0.0**. Its modular routing structure separates product management, inventory information, forecasting, inventory intelligence, recommendations, analytics, optimization candidates, decision processing, stock movements, conversational assistance, weather services, decision workflows, and marketing integration.

A total of 14 API routers are registered in the main application. Each router provides an interface to its associated backend functionality, allowing the Flutter Web frontend and other integrated components to communicate with the relevant services.

In addition to the registered routers, the application provides a root endpoint (GET /) that returns the service name and version, and a health endpoint (GET /health) that returns a basic application status response.

Table 5.2 summarises the registered backend router modules and their principal responsibilities.

Table 5.2: Backend module summary

Module	Registered Router	Responsibility
Products	product_routes	Provides API access to product-related information and supporting product data.
Inventory	inventory_routes	Provides access to current inventory information.
Stores	store_routes	Provides store-related information required by the application.
Forecasting	forecast_routes	Provides forecasting operations for supported demand prediction horizons.
Inventory Intelligence	intelligence_routes	Exposes inventory assessment and stock intelligence functionality.
Recommendations	recommendation_routes	Provides access to inventory recommendation functionality.
Analytics	analytics_routes	Exposes inventory and business analytics information.
Optimization Candidates	optimization_candidate_routes	Provides access to generated optimization candidates and associated recommendation information.

Module	Registered Router	Responsibility
Decision Engine	decision_engine_routes	Exposes decision-processing functionality used by the inventory optimization system.
Stock Movements	stock_movement_routes	Supports stock movement processing and related operational workflows.
Manager Assistant	chat_routes	Provides the conversational interface for inventory-related questions.
Weather Integration	weather_routes	Exposes weather-related functionality supporting forecasting services.
Decision Workflows	decision_workflow_routes	Provides interfaces for decision workflow operations.
Marketing Integration	marketing_opportunity_routes	Provides interfaces for marketing opportunity information and cross-component integration.

The modular backend structure separates application responsibilities into dedicated API routers while allowing them to share common services and database resources. This organisation supports the integration of demand forecasting, inventory intelligence, stock optimization, movement processing, analytics, and conversational assistance within a single backend application.

5.3. Integration with the Other Components

The Smart Omni-Retail System consists of multiple interconnected research components that share product, store, and inventory-related information. The AI-Driven Smart Inventory and Stock Flow Optimization component was developed to support integration with the Smart Mirror and Personalized Marketing Intelligence components through shared data structures and backend services.

The integration work undertaken as part of this individual research focused on product information, store information, product image references, inventory availability, and

promotional opportunity sharing. The implementation and evaluation of the other research components remained outside the scope of this individual contribution.

Figure 5.2 illustrates the integration between the Smart Inventory component, the Smart Mirror component, the Personalized Marketing Intelligence component, and the shared system infrastructure. The diagram distinguishes implemented integration pathways from partially implemented functionality.

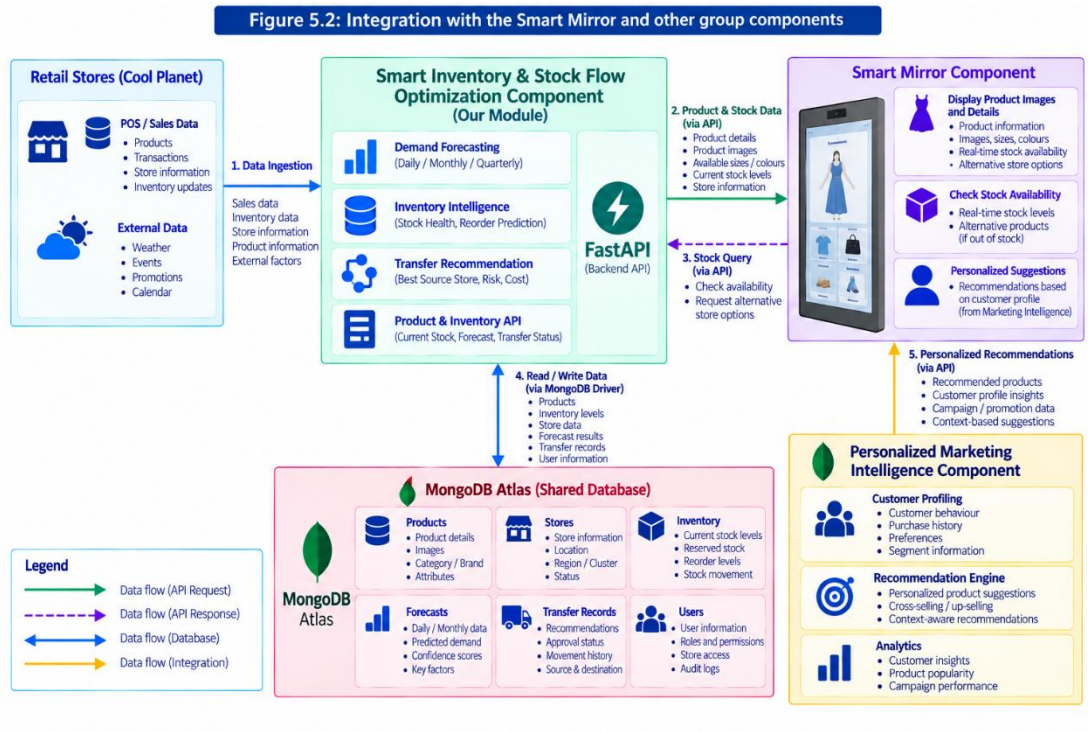


Figure 5.2: Integration with the Smart Mirror and other group components

Product image support was implemented to facilitate integration with the Smart Mirror component. Instead of storing large image files directly in MongoDB, representative product images were generated and hosted using GitHub Pages. This approach reduced the storage requirements of the operational database and avoided the additional billing requirements associated with Firebase Storage.

Each product record contains image-related metadata, including image_key, image_url, image_type, and image_source. The image_type field identifies the generated images as REPRESENTATIVE, while the image_source field identifies GITHUB_PAGES as the hosting source. The Smart Mirror component can use these

image URLs to retrieve and display representative product images alongside the relevant product information.

The generated images are representative illustrations rather than actual photographs of the corresponding retail products. Therefore, the image integration was implemented as a practical supporting mechanism for the overall system rather than as a separate research contribution.

Integration with the Personalized Marketing Intelligence component was designed to support the exchange of inventory-related information for marketing decision-making. The implemented Inventory-to-Marketing integration enables the Smart Inventory component to share identified promotional opportunities together with associated product, store, and stock information. This allows the Marketing component to consider inventory conditions when identifying potential promotional activities.

A reverse Marketing-to-Inventory integration was also planned to support stock feasibility assessments for proposed marketing campaigns. Under this workflow, the Marketing component submits a request containing the relevant product, store, quantity, and campaign information. The Smart Inventory component evaluates the available inventory and returns an assessment indicating whether sufficient stock is available to support the proposed campaign.

At the time of this report, the reverse request-and-response integration was partially implemented and remained under development. Therefore, successful end-to-end operation of this workflow has not been claimed. The implemented Inventory-to-Marketing opportunity-sharing functionality and the partially completed Marketing-to-Inventory feasibility assessment functionality are distinguished in Figure 5.2.

MongoDB Atlas provides shared data storage for product, store, and inventory-related information, while Firebase Authentication supports the common authentication infrastructure. These shared technologies help maintain consistency between the research components and reduce unnecessary duplication of reference data.

Overall, the integration work demonstrates how the Smart Inventory component contributes to the wider Smart Omni-Retail System by providing product information, representative product images, inventory context, and promotional opportunities. The

remaining Marketing-to-Inventory feasibility integration requires further implementation and testing before its operational effectiveness can be confirmed.

5.3.1. Campaign Feasibility Request and Response Contract

The integration between the Smart Inventory and Personalized Marketing Intelligence components includes a proposed request-and-response mechanism for assessing the inventory feasibility of marketing campaigns. This mechanism is intended to allow the Marketing component to request a stock availability assessment before proceeding with a proposed promotional activity.

Under the proposed integration contract, the Marketing component submits a feasibility request containing a campaign identifier, relevant product identifiers, target store identifiers, proposed campaign dates, and expected campaign quantities. The Smart Inventory component is responsible for validating the request, retrieving the relevant inventory records and demand forecasting information, and evaluating whether the requested quantities can be supported by the available stock.

The expected response includes an assessment of campaign feasibility, supporting quantities for each relevant product and store, reasons for the assessment, a request identifier, and an assessment timestamp. The response should distinguish between campaigns that can be supported, campaigns with insufficient stock, and situations where an assessment cannot be reliably completed because inventory information is missing or outdated.

The feasibility assessment is intended to provide advisory decision support rather than automatic campaign approval. A supported assessment indicates that sufficient stock is available under the assumptions used during evaluation. It does not guarantee future inventory availability or automatically authorize the proposed campaign.

Before the integration is finalized, the participating components must establish consistent rules for interpreting campaign quantities, including whether the requested quantities represent additional promotional demand, total expected demand, or stock reserved for the campaign. The treatment of incoming stock transfers and the possibility of reserving inventory must also be defined. Appropriate authentication and

authorization controls are required to restrict access to campaign information and feasibility assessments.

At the time of this report, the reverse Marketing-to-Inventory integration was partially implemented. Therefore, the request-and-response contract described in this section represents the intended integration specification rather than a fully verified operational API.

5.3.2. Planned End-to-End Verification Scenarios

End-to-end integration testing is required to verify the exchange of information between the Smart Inventory and Personalized Marketing Intelligence components. The planned verification activities cover both the implemented Inventory-to-Marketing opportunity-sharing functionality and the partially implemented Marketing-to-Inventory feasibility assessment workflow.

The planned integration verification will cover the following scenarios:

1. **Promotional Opportunity Submission:** Verify that a promotional opportunity identified by the Smart Inventory component can be successfully submitted to the Personalized Marketing Intelligence component with the relevant product, store, and inventory information.
2. **Campaign Feasibility Request Validation:** Verify that the Smart Inventory component can receive and validate a campaign feasibility request submitted by the Marketing component, including the required product identifiers, store identifiers, campaign dates, and expected quantities.
3. **Inventory Availability Assessment:** Verify that the system retrieves the appropriate inventory records and evaluates stock availability for the requested products and store locations.
4. **Feasibility Response Generation:** Verify that the system returns an appropriate supported or not-supported assessment, including relevant stock quantities and reasons for the result.
5. **Invalid and Unavailable Data Handling:** Verify that incomplete requests, invalid identifiers, missing inventory records, and outdated stock information

are rejected or handled safely without producing unsupported feasibility conclusions.

The verification process will also consider the consistency of request identifiers, the association of assessment results with the correct campaigns, and the handling of repeated requests. Relevant request and response payloads, API response statuses, stock values, execution dates, and observed results will be recorded as supporting evidence during testing.

At the time of this report, only the Inventory-to-Marketing promotional opportunity-sharing functionality can be described as implemented. Regression testing of this functionality remains pending, while the reverse Marketing-to-Inventory feasibility assessment scenarios require further implementation and end-to-end verification.

These planned verification activities will support the evaluation of integration correctness, data consistency, and reliability. The corresponding test results and supporting evidence will be documented in Chapter 6 after the relevant testing activities have been completed.

5.4. Verification and Software Testing

Software testing was conducted to verify the functionality and integration of the major modules within the AI-Driven Smart Inventory and Stock Flow Optimization component. The testing approach was primarily scenario-based, focusing on whether the implemented application functions produced the expected outputs and supported the required inventory management workflows.

The verification activities covered demand forecasting, inventory intelligence, stock transfer optimization, recommendation generation, approval workflows, stock movement execution, inventory synchronization, and transaction history. Additional functional checks concerned explainable AI, the Manager Assistant, and product image integration.

The researcher conducted functional testing using the synthetic dataset and inventory records maintained in MongoDB Atlas. The tests involved executing application operations, examining the returned results, and checking relevant inventory and

movement records where applicable. These activities helped assess whether the individual modules operated as intended and whether information was transferred consistently between related services.

Table 5.3 summarises the principal areas covered by functional verification.

Table 5.3: Test-case summary

ID	Area	Scenario verified
TC01	Demand Forecasting	Generation of daily, monthly, and quarterly demand forecasts for selected products and stores.
TC02	Inventory Intelligence	Generation of stock health classifications, days-on-hand estimates, replenishment information, and overstock assessments.
TC03	Stock Optimization	Identification of potential stock transfers and generation of recommendations with alternative source stores.
TC04	Explainable AI and Manager Assistant	Generation of inventory-related explanations and conversational responses using available application information.
TC05	Stock Movement Workflow	Processing of recommendation approval, rejection, cancellation, and movement execution operations.
TC06	Inventory Synchronization	Verification of source and destination inventory quantity changes and the creation of corresponding transaction records.
TC07	System Integration	Verification of product image references and the interaction between forecasting, inventory intelligence, optimization, and stock movement modules.

The end-to-end stock movement scenario was particularly important because it involved interactions between multiple backend services. The workflow began with demand forecasting and inventory intelligence, followed by the generation of a stock transfer recommendation and its associated explanation. After managerial approval, the stock movement was executed, and the corresponding inventory information was updated.

During the reported execution tests, the researcher checked the stock quantities at both the source and destination stores to confirm that the transfer was reflected in the relevant inventory records. The associated transaction history and movement status information were also examined to verify that the completed operation could be traced through the application.

Testing of the implemented Inventory functions indicated that the principal forecasting, recommendation, approval, execution, and transaction-history workflows operated as expected in the prototype environment. However, these functional

observations do not establish performance under production workloads or guarantee the absence of software defects.

The integration with the Personalized Marketing Intelligence component was evaluated separately according to its implementation status. Although the Inventory-to-Marketing promotional opportunity-sharing functionality was implemented, its regression testing remained pending. The reverse Marketing-to-Inventory campaign feasibility workflow was partially implemented and had not completed confirmed end-to-end testing. Consequently, this functionality is excluded from claims of successfully completed integration testing.

The verification activities were conducted using the research prototype and synthetic operational data. The detailed test scenarios, observed outputs, and supporting evidence are addressed in the system evaluation presented in Chapter 6.

5.5. Deployment and Operation

The AI-Driven Smart Inventory and Stock Flow Optimization component was deployed using a cloud-based architecture to support access to its backend services and operational database. The backend application was developed using FastAPI and deployed on Microsoft Azure, while MongoDB Atlas was used to host the operational database.

The Flutter Web application communicates with the FastAPI backend through RESTful API endpoints. The backend processes requests from the frontend, performs the required business logic, retrieves or updates database records, and returns the corresponding results. This architecture separates the user interface from the backend processing and data storage layers.

The deployed backend provides functionality for demand forecasting, inventory intelligence, stock transfer optimization, stock movement management, analytics, and the Manager Assistant. The FastAPI application registers separate API routers for these functional areas, supporting a modular service structure.

The demand forecasting services use trained XGBoost model artifacts saved during the offline model development process. These artifacts are loaded by the relevant

forecasting services to generate predictions without retraining the models for each request.

MongoDB Atlas stores the operational data required by the application, including product and store information, current inventory, optimization candidates, stock movement records, inventory transactions, and analytics-related information. The use of a managed cloud database allows the backend to access persistent operational information independently of the application execution environment.

The system also depends on external services for specific supporting functions. Firebase Authentication provides authentication functionality, the Groq API supports AI-generated explanations and conversational responses, and the Open-Meteo API supplies weather-related information for forecasting services.

The weather integration incorporates temporary in-memory caching to reduce repeated requests to the external API. Short-range weather forecasts are cached for 30 minutes, while historical seasonal weather profiles are cached for 24 hours. These caching mechanisms support more efficient retrieval of weather-related information during application operation.

The backend application includes a root endpoint that returns the application name and version, together with a basic health endpoint that returns the configured application status. These endpoints support basic service identification and availability checks.

The deployment represents a functional research prototype developed and tested using synthetic retail data. Although the system demonstrates the integration of forecasting, inventory intelligence, optimization, and stock movement management in a cloud-based environment, it has not been validated through sustained operation across live retail stores.

Therefore, the deployment demonstrates the technical feasibility of the proposed approach within the research environment rather than establishing production-level reliability, scalability, or operational effectiveness under real retail conditions.

6. RESULTS AND VALIDATION

6.1. Research Results

6.1.1. Forecasting performance

The demand forecasting performance of the proposed Smart Inventory system was evaluated using XGBoost regression models trained on a fully synthetic retail dataset generated through Python scripts. Separate models were developed for daily and monthly demand forecasting to accommodate differences in demand patterns and prediction horizons.

The model evaluations recorded in the original Google Colab notebooks used a chronological training and testing approach. Data generated for the period before 1 January 2025 was used for training, while observations from 2025 were reserved for testing. This approach was intended to evaluate the models using observations from a later period rather than a randomly selected subset of the dataset.

Three regression evaluation metrics were considered: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2). MAE measures the average absolute difference between actual and predicted demand, while RMSE places greater emphasis on larger prediction errors. R^2 indicates the proportion of variation in the evaluated demand values explained by the model.

The daily forecasting model recorded an MAE of 0.5418 units, an RMSE of 0.9436 units, and an R^2 value of 0.9497. The monthly forecasting model recorded an MAE of 14.2128 units, an RMSE of 26.3061 units, and an R^2 value of 0.9353.

Table 6.1: Forecast evaluation results

Model	MAE	RMSE	R^2	RMSE / MAE
Daily	0.5418	0.9436	0.9497	1.74
Monthly	14.213	26.306	0.9353	1.85

These results represent the evaluation outputs recorded during model development. They have not yet been independently reproduced using an audited feature-engineering and evaluation pipeline. In particular, the construction of grouped historical and rolling demand features requires further examination to exclude possible information leakage.

Therefore, the reported metrics should be interpreted as preliminary evidence of predictive performance on the synthetic test dataset.

Quarterly forecasting was implemented by generating three sequential monthly predictions and aggregating their results. Since a separate quarterly forecasting model was not trained or independently evaluated, no quarterly MAE, RMSE, or R^2 value is reported.

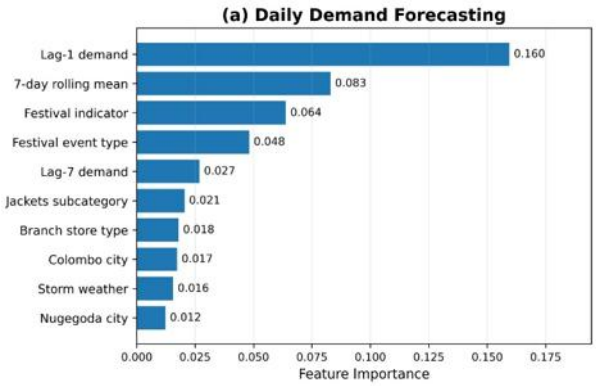


Figure 6.1: Daily forecasting performance visualization

Figure 6.1 is reserved for the visual evaluation of the daily forecasting model. The actual and predicted demand values will be compared to illustrate the model's prediction behaviour over the selected test period. The residual distribution and actual-versus-predicted scatter plot will provide additional information about the magnitude and distribution of prediction errors.

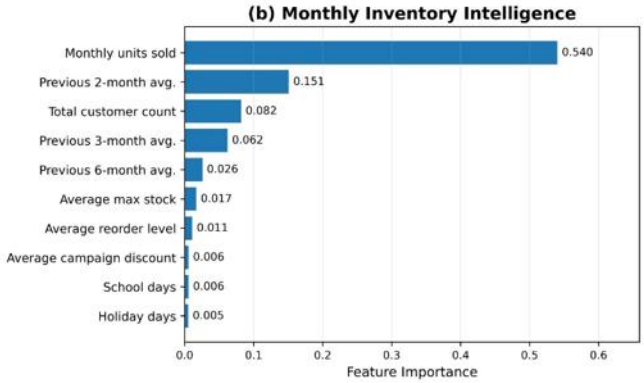


Figure 6.2: Monthly forecasting performance visualization

Figure 6.2 is reserved for the corresponding monthly forecasting evaluation. It will illustrate the relationship between actual and predicted monthly demand and support examination of seasonal variations across the selected evaluation period.

Both figures will be generated using authentic prediction outputs from the original held-out evaluations or a documented reproducible evaluation run. Until these outputs are verified, no specific conclusions are drawn regarding forecasting errors during promotional periods, seasonal demand peaks, or individual product categories.

The daily model's reported MAE indicates an average absolute prediction error of approximately 0.54 units per evaluated daily observation. The monthly model's MAE indicates an average absolute error of approximately 14.21 units per evaluated monthly observation. These values should not be compared directly because daily and monthly predictions represent different aggregation periods and demand scales.

The reported R^2 values suggest that both models captured a substantial proportion of demand variation within their respective synthetic evaluation datasets. However, the results do not establish equivalent predictive accuracy when applied to actual retail sales records.

Furthermore, the RMSE values are higher than the corresponding MAE values, indicating that some prediction errors may be larger than the average absolute error. Additional residual analysis would be required to determine whether these errors are associated with specific stores, products, demand patterns, or seasonal conditions.

Overall, the recorded evaluation results provide preliminary support for the feasibility of using XGBoost models for daily and monthly retail demand forecasting. Nevertheless, validation using real retail data, comparison against baseline forecasting methods, and further examination of the feature-engineering pipeline would be required before drawing stronger conclusions about generalisation and operational forecasting accuracy.

6.2. Product and Technical Results

The AI-Driven Smart Inventory and Stock Flow Optimization component was implemented as a web-based inventory decision-support application integrating

machine learning, inventory analysis, stock transfer optimization, and operational stock movement management.

The completed application contains seven main Flutter Web modules: Dashboard, Forecasting, Inventory Intelligence, Optimization, Stock Movements, Analytics, and Manager Assistant. These modules communicate with the FastAPI backend, which processes forecasting requests, evaluates inventory conditions, generates transfer recommendations, manages movement workflows, and retrieves operational information from MongoDB Atlas.

The implemented system supports the identification of potential inventory shortages and overstock situations, evaluation of alternative source stores, generation of transfer recommendations, and execution of approved stock movements.

Functional testing conducted during development indicated that the principal inventory workflows operated as expected in the prototype environment. The researcher reported successful execution of forecasting, inventory intelligence, recommendation, approval, and stock movement operations, including updates to source and destination inventory quantities and the creation of transaction records.

The following figures are reserved for documenting a representative stock transfer scenario using actual application outputs. Where possible, the same product, source store, destination store, and movement record should be used across Figures 6.3, 6.4, and 6.5 to demonstrate the complete workflow.

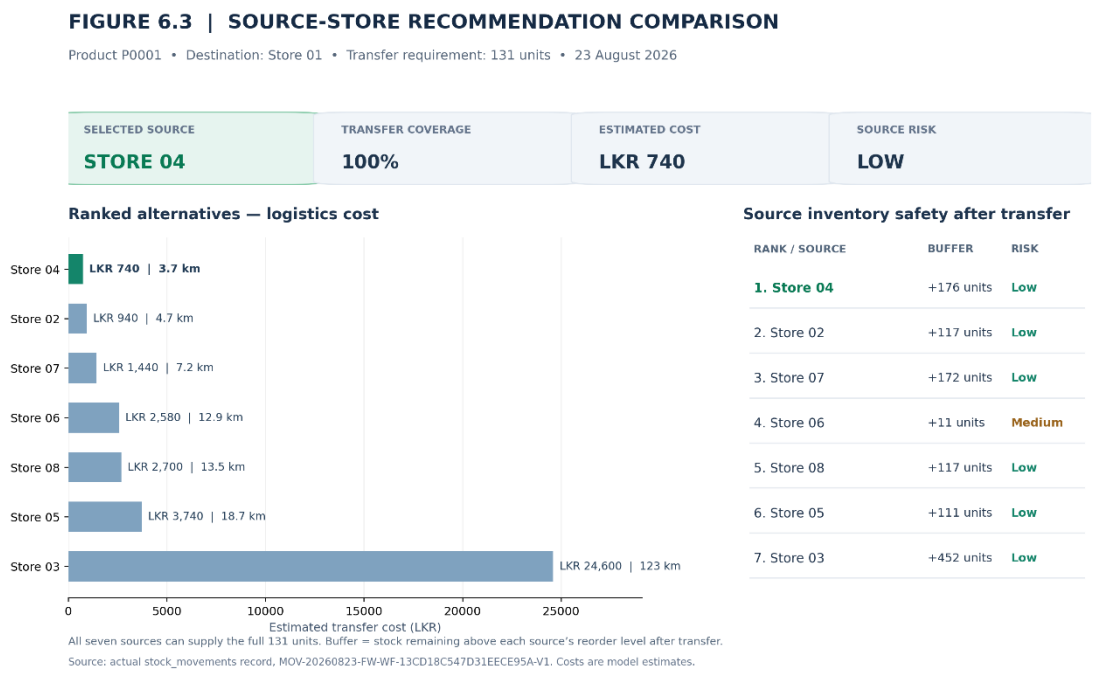


Figure 6.3: Example recommendation and alternative-store comparison

Figure 6.3 will demonstrate how the Optimization module presents the selected source store and alternative transfer options. The recommendation information includes the proposed transfer quantity, destination stock coverage, remaining source inventory, distance, estimated logistics cost, risk level, and confidence indicator.

These factors are intended to support managerial evaluation of the proposed transfer rather than automatically execute the recommendation.

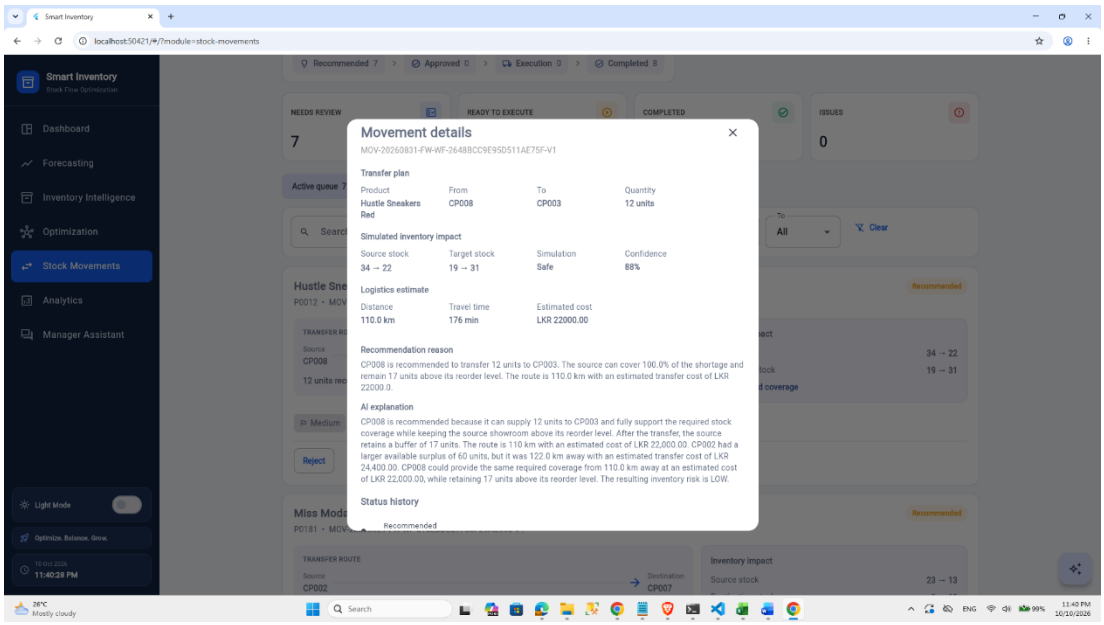


Figure 6.4: Actual Application Output Showing the Explainable Stock Transfer Recommendation, Inventory Impact, and Logistics Evaluation.

Figure 6.4 will illustrate how the Explainable AI layer presents the reasoning associated with a stock transfer recommendation. The explanation is generated using structured recommendation information, allowing managers to interpret the factors influencing the selected source store and the differences between available alternatives.

The recommendation calculations and ranking decisions are performed through deterministic backend logic. The Groq-based language model is used to present these results in natural language and does not independently determine transfer quantities, modify inventory records, or authorize stock movements.

This separation supports the traceability of recommendation decisions. However, the architecture alone does not guarantee that every generated explanation is factually accurate. A documented review of generated explanations against their corresponding structured input data would be required to establish the consistency and reliability of the explanation outputs.

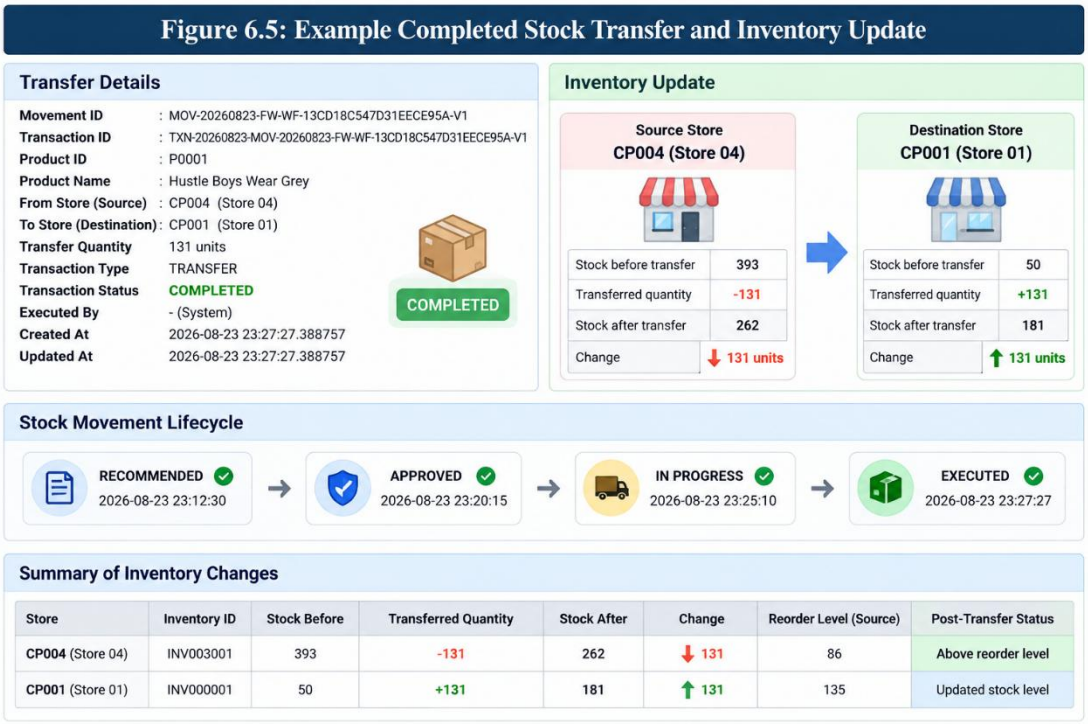


Figure 6.5: Example recommendation and alternative-store comparison

Figure 6.5 illustrates the successful execution of a stock transfer recommendation for Product P0001 between Store 04 (CP004) and Store 01 (CP001). Following the recommendation and approval stages, the system executed a transfer of 131 units.

The recorded inventory quantities show that the source store's stock decreased from 393 to 262 units, while the destination store's stock increased from 50 to 181 units. The source store remained above its configured reorder level of 86 units, preserving an additional buffer of 176 units.

The movement history recorded the progression through RECOMMENDED, APPROVED, IN_PROGRESS, and EXECUTED states. These results demonstrate the implementation of the stock transfer workflow and the corresponding inventory quantity updates in the developed system.

Figure 6.5 will demonstrate the completion of an approved stock transfer and its effect on inventory records. The evidence should show the movement status, transferred quantity, source and destination inventory balances before and after execution, and the corresponding inventory transaction.

This scenario is particularly important because it demonstrates the connection between predictive decision support and operational inventory management. A transfer recommendation becomes an inventory movement only after the relevant workflow operations have been completed.

The movement history and transaction records provide a basis for tracing completed transfers and examining changes to inventory quantities.

The final evaluation will include a documented transfer example containing the actual product identifier, source and destination stores, recommended quantity, movement identifier, pre-transfer and post-transfer inventory balances, and associated transaction information. These details will be taken from genuine application records rather than illustrative or manually fabricated values.

Overall, the implemented prototype demonstrates the technical integration of demand forecasting, inventory intelligence, transfer optimization, explainable recommendations, and stock movement management within a unified application. The reported functional results support the feasibility of the proposed implementation within the synthetic research environment, but they do not establish measurable improvements in real-world retail inventory performance.

6.3. User or Stakeholder Validation

Stakeholder feedback was considered during the development and refinement of the Smart Inventory and Stock Flow Optimization component. Informal feedback was obtained from individuals familiar with retail inventory management and related operational processes, together with guidance received through university supervision.

The stakeholder roles represented in these discussions included Inventory Management, Merchandise Planning, IT Management, Sales Assistance, and Cashier functions. These roles were relevant to the research because they are associated with different aspects of retail operations, including inventory monitoring, replenishment planning, stock availability enquiries, merchandise decisions, and system support.

The feedback was considered during iterative development to improve the practical relevance of the application and its presentation of inventory-related information. In

particular, the research sought to provide a system that could present forecasting results, stock conditions, and transfer recommendations in a form understandable to operational users and managers.

The validation approach was informal and qualitative. It did not involve a controlled usability experiment, a standardised questionnaire, or a statistically representative sample of retail employees. Furthermore, the available records do not establish a complete participant count, a common set of evaluation tasks, or quantitative measures of user satisfaction.

Consequently, no numerical usability score, user acceptance percentage, task completion improvement, or statistically significant stakeholder outcome is claimed.

Table 6.2: Stakeholder consultation and evidence status

Evidence type	What to report
Inventory Management	Inventory availability, stock monitoring, shortage identification, and replenishment-related decision support.
Merchandise Planning	Demand forecasting, product movement, stock distribution, and merchandise-related planning.
IT Management	Application functionality, technical integration, data handling, and system maintainability.
Sales Assistance	Product availability enquiries and store-level inventory information.
Cashier	Retail transaction context and product availability considerations.
University Supervision	Academic guidance, research scope, implementation review, and presentation of research findings.

The stakeholder consultation provided qualitative input relevant to the design and implementation of the prototype. However, informal feedback alone cannot establish that the proposed system improves stock availability, reduces inventory costs, or increases operational efficiency.

Such outcomes would require structured evaluation using actual retail operations, clearly defined performance indicators, and an appropriate comparison with existing inventory management practices.

Therefore, the stakeholder feedback is interpreted as supporting the practical relevance of the research problem and the prototype design rather than providing independent proof of operational effectiveness.

6.4. Integration Results

The integration of the AI-Driven Smart Inventory and Stock Flow Optimization component with the other components of the Smart Omni-Retail System was evaluated based on the functionality implemented and the corresponding level of verification. The primary integration activities involved sharing product information, product images, inventory availability, and promotional opportunities with the Smart Mirror and Personalized Marketing Intelligence components.

The integration with the **Smart Mirror component** was implemented to provide access to product-related information, representative product images, and inventory availability. Product image metadata, including image identifiers and hosted image URLs, enables the Smart Mirror to retrieve and display the relevant product images. The integration also supports access to inventory information associated with the selected products.

The researcher reports that the Smart Mirror integration was successfully implemented and tested during system development. This demonstrates the ability of the Smart Inventory component to provide product and inventory information to another component of the overall research system. However, the corresponding interface screenshots and API response evidence are to be included in the final thesis to support the reported integration results.

The integration with the **Personalized Marketing Intelligence component** was developed to support inventory-aware marketing decisions. The implemented Inventory-to-Marketing functionality enables the Smart Inventory component to share identified promotional opportunities together with relevant product, store, and stock information. This information can be used by the Marketing component when evaluating potential promotional activities.

A reverse Marketing-to-Inventory integration was also designed to allow the Marketing component to request stock feasibility assessments for proposed campaigns. The intended workflow involves submitting campaign requirements, evaluating inventory availability against the requested quantities, and returning an assessment with supporting stock information.

At the time of this evaluation, the reverse request-and-response functionality was partially implemented and had not completed confirmed end-to-end integration testing. Therefore, the complete bidirectional Marketing integration cannot be considered fully validated. The remaining development and testing activities are necessary before the feasibility assessment workflow can be evaluated as an operational integration.

Furthermore, the proposed inventory feasibility assessment is intended to provide advisory information for marketing planning. It does not automatically approve campaigns, reserve inventory, or initiate campaign execution.

Overall, the integration results demonstrate that the Smart Inventory component supports information sharing within the Smart Omni-Retail System. The Smart Mirror integration was reported as implemented and tested, while the Inventory-to-Marketing promotional opportunity-sharing functionality was implemented. The reverse Marketing-to-Inventory feasibility assessment remains partially implemented and requires further verification.

6.5. Summary of Findings

The results presented in this chapter demonstrate the implementation and preliminary evaluation of the AI-Driven Smart Inventory and Stock Flow Optimization component. The research combined machine learning-based demand forecasting, inventory intelligence, deterministic stock transfer optimization, explainable AI, and operational stock movement management within a unified web-based application.

The demand forecasting evaluation recorded promising results for the daily and monthly XGBoost models using the fully synthetic retail dataset. The daily model achieved a reported R^2 value of 0.9497, while the monthly model achieved an R^2 value of 0.9353. These results indicate that the models captured a substantial proportion of demand variation within their respective synthetic evaluation datasets. However, the reported performance remains subject to further examination of the feature-engineering process and independent reproduction of the evaluation results. The findings therefore cannot be directly generalized to actual retail sales environments.

The implemented system demonstrates the integration of forecasting results with inventory assessment and stock transfer decision-making. The Inventory Intelligence module supports the identification of shortage and overstock conditions, while the Optimization module evaluates potential source stores and generates recommendations using defined inventory and logistics-related factors.

The Explainable AI layer provides natural-language interpretations of forecasting and recommendation results, supporting managerial understanding of the generated outputs. The Stock Movement module connects these recommendations to operational workflows through approval, execution, inventory quantity updates, and transaction recording.

Functional testing conducted during development indicated that the principal inventory management workflows operated as expected within the prototype environment. In particular, the researcher reported successful stock movement execution involving updates to source and destination inventory quantities and the creation of corresponding transaction records. However, the evaluation does not establish a quantified software test pass rate or independently verified production-level reliability.

Informal stakeholder feedback was considered during development to improve the practical relevance of the proposed system. Although the consultation involved individuals familiar with retail operations and technical management, no formal usability experiment or quantitative user satisfaction assessment was conducted. Consequently, the feedback is treated as qualitative input rather than statistical evidence of system effectiveness.

The integration results further demonstrate the role of the Smart Inventory component within the wider Smart Omni-Retail System. Product information, representative product images, and inventory availability were shared with the Smart Mirror component, and the integration was reported as successfully tested. Promotional opportunity-sharing functionality was implemented for the Personalized Marketing Intelligence component, while the reverse campaign feasibility assessment workflow remained partially implemented and required further testing.

Overall, the findings support the **technical feasibility of the proposed Smart Inventory and Stock Flow Optimization approach within a research prototype environment**. The system demonstrates how predictive analytics, deterministic inventory optimization, explainable decision support, and controlled stock movement workflows can be combined into an integrated application.

Nevertheless, further evaluation using real retail data, independently reproduced forecasting results, comprehensive integration testing, and structured stakeholder assessment would be necessary before making stronger claims regarding forecasting generalization, operational efficiency, cost reduction, or readiness for deployment in a live retail environment.

7. DISCUSSION

7.1. Interpretation of Findings

The recorded XGBoost results indicate that the daily and monthly forecasting models learned regularities within the team-generated synthetic retail dataset. The original notebooks report daily MAE 0.5418, RMSE 0.9436 and R^2 0.9497, and monthly MAE 14.2128, RMSE 26.3061 and R^2 0.9353, using a chronological holdout for 2025. These are notebook-reported values, not independently reproduced results. Because the Python-generated data were not calibrated against confidential real retail sales, the reported fit establishes feasibility under the synthetic assumptions rather than forecasting accuracy in an operating retailer. The unresolved audit of grouped rolling-feature calculations further limits confidence in the current evaluation figures.

The daily and monthly models operate at different aggregation levels, while the application also offers multi-day and quarterly forecasts. The one-step daily holdout result does not by itself validate recursive seven-day accuracy; similarly, the monthly holdout does not establish separate quarterly accuracy. Lagged and rolling demand features can be informative, but group boundaries and the order of feature construction relative to the temporal split must be checked to rule out contamination. A corrected, reproducible feature pipeline, naive baselines and horizon-specific backtesting are necessary before making comparative performance claims.

For inventory decisions, forecast error is consequential because underprediction can cause shortages and overprediction can lead to excess stock or unnecessary movement. The higher RMSE than MAE indicates that larger errors receive more weight in the squared-error measure, but no causal explanation for those errors is established by the reported aggregate metrics alone. The design therefore uses forecasts as decision-support inputs and retains a human approval stage for stock transfers. Heuristic confidence labels in the application must not be interpreted as statistically calibrated probabilities.

The implementation results address a different research question: whether forecasting, stock assessment, ranked source recommendations, explanations and controlled transfer actions can operate together. The researcher reports successful tests of the

major flows, including updates to source and destination inventory and transaction logs. Until the complete case-by-case test record and actual transaction evidence are attached, these remain reported functional observations rather than independently audited outcomes. They do not demonstrate an optimal transfer policy, measured savings or improvements in live store operations.

7.2. Comparison with Related Work

Published retail forecasting studies, including the M5 competition [2], employ different data-generating conditions, targets and evaluation metrics. Their reported scores cannot be compared numerically with the MAE or R^2 obtained from this fully synthetic dataset. XGBoost is an established choice for structured tabular predictors [5], and using calendar, promotion and weather features is consistent with the forecasting literature [1], [15]. However, the thesis does not establish superiority over a seasonal naive forecast, moving average or another model because no independently verified baseline experiment is available. This is a limitation of comparative evidence, not evidence that alternative approaches perform worse.

In inventory optimisation, transshipment research distinguishes proactive and reactive transfers [3]. This component combines forecast-informed shortage detection with a deterministic ranking of candidate source stores, considering available surplus, coverage, distance, estimated logistics cost and risk. Its rules prioritise interpretability and operational feasibility; they do not solve a globally optimal multi-store transportation problem. Work on fast-fashion transshipment under capacity constraints [6] offers a relevant comparison of problem setting, but its algorithm was neither implemented nor benchmarked here. Accordingly, the thesis claims an integrated decision-support design, not a demonstrated improvement over mathematical optimisation methods.

For explainability, model-attribution methods such as SHAP explain contributions to a model prediction [7], [8], whereas the language layer in this system communicates the reasons and alternatives produced by a deterministic recommendation engine. These are distinct explanatory tasks. The design supports contrastive managerial questions—why a source store was preferred over another—consistent with the role of

contrastive explanations discussed by Miller [13]. To limit unsupported decision-making, the language model receives precomputed decision facts and cannot authoritatively alter the transfer ranking. This architectural restriction does not guarantee factual correctness of every generated sentence; no formal explanation-fidelity or hallucination-rate experiment was performed [9], [14].

Relative to the separate forecasting, inventory monitoring and stock-transfer approaches reviewed in Chapter 2, the practical contribution is their combination in a single auditable workflow. The novelty claim concerns the system-level arrangement and its application to the defined multi-branch fashion-retail scenario, rather than invention of XGBoost, transfer heuristics or large-language-model explanations.

7.3. Research Contribution

The principal research contribution is the design and preliminary assessment of a hybrid inventory decision-support architecture. A deterministic decision engine calculates stock health, source ranking and movement quantities, while a language model is restricted to presenting an explanation of those results. The design also incorporates a controlled movement lifecycle, source/destination stock updates, transaction records, fallback behaviour for AI explanations and an operational current-state inventory representation in MongoDB Atlas. These are architecture and implementation contributions; no controlled comparison demonstrates that this architecture outperforms competing solutions. Table 7.1 identifies the supporting evidence and remaining validation gaps.

Table 7.1: Summary of research contributions and strength of evidence

Contribution	Description	Evidence available
Python-generated synthetic dataset	Five-year multi-store synthetic retail dataset generated by the research team, without calibration to confidential real sales	Dataset files and original Colab notebooks; generator code/rules pending attachment
Forecasting evaluation	XGBoost daily and monthly models with lag, calendar, price, promotion and weather features	Original notebook-reported daily/monthly metrics; independent feature audit and baseline pending

Contribution	Description	Evidence available
Deterministic transfer engine	Ranked source selection with coverage, safety stock, distance, cost and risk	Deterministic service implementation; worked transaction-level example pending
Hybrid explainable AI	Language model explains, and the engine decides, with cache and fallback	Implementation and architectural restriction; explanation-fidelity evaluation not quantified
Controlled movement workflow	Lifecycle with inventory synchronization and duplicate-execution prevention	Researcher-reported functional tests; consolidated case log and stock balance evidence pending
Data-model decision	Current-state MongoDB design instead of full history	MongoDB Atlas current-state design; no measured performance or cost comparison

7.4. Product Contribution

The researcher developed all seven Inventory application modules: Dashboard, Forecasting, Inventory Intelligence, Optimization, Stock Movements, Analytics and Manager Assistant. Together they present forecast outputs, inventory health indicators, ranked recommendations, transfer decisions, audit-oriented records and conversational access to inventory information. The supporting implementation uses Flutter Web, a FastAPI backend deployed on Azure, and MongoDB Atlas. Smart Mirror receives product information and inventory availability as part of the integrated group system. Inventory-to-Marketing opportunity sharing is implemented; the reverse campaign-feasibility request and response remains partially implemented. The component is a research prototype, not a validated production installation at retail branches, and other group components must not be attributed to the individual researcher.

7.5. Limitations and Threats to Validity

Threats to validity arise at several levels. External validity is limited because the five-year, eight-store dataset was generated using Python by the research team without access to confidential real sales patterns. Internal validity remains subject to the audit of grouped lag and rolling calculations and independent metric reproduction. Construct validity is limited because MAE, RMSE and R^2 assess predictive fit, not stock-out reduction, financial return or recommendation quality. Functional validity is

constrained by the absence of a consolidated test log and a recorded worked transfer example. Informal feedback from Inventory Managers, Merchandise Planners, IT Managers, Sales Assistants, Cashiers and university supervision provides practitioner perspectives but not representative quantitative usability evidence. The incomplete reverse Marketing integration is a system-completeness limitation, not a completed outcome. Table 7.2 summarises these boundaries.

Table 7.2: Limitations and their effects

Limitation	Effect	Where discussed
Fully synthetic dataset generated without real-sales calibration	External validity of forecast metrics for live retail remains unknown	Sections 3.2, 7.1
Limited live ERP integration	Stock data is not synchronised with real point-of-sale or ERP systems	Section 1.7
Model performance depends on data quality	Errors, gaps or shifts in real data would reduce accuracy	Section 7.1
Forecast uncertainty grows with the horizon	Recursive and quarterly accuracy have not been independently evaluated	Section 7.1
Simplified logistics cost estimation	Cost ranking may differ from real transport costs	Section 4.4.3
External Groq and Open-Meteo services depend on availability, quota or connectivity	Explanations, chat answers or live weather inputs may be delayed, and the system may fall back to local text or an alternative weather value	Section 4.4.4
Explanations aid readability but do not replace business logic	The layer must not be treated as a source of decisions	Section 4.3
Informal stakeholder consultation without standardised evaluation	No quantified usability, task success or representative user evidence	Section 6.3
Representative images are not product-specific photographs	The Smart Mirror shows illustrative images only	Section 5.3
No production deployment across real stores	Operational behaviour at scale is unverified	Section 5.5
Ranking is not a global optimum	Transfers for different products and stores are not jointly optimised	Section 4.3

7.6. Implications

For retail practice, the prototype illustrates how forecast estimates, stock rules, alternative-store comparisons and approval-controlled transfers can be presented in one workflow. It does not establish that deploying the prototype would reduce costs or

stock-outs. For system design, the work demonstrates a useful separation between auditable business logic and natural-language communication. For future evaluation, the priorities are to reproduce the forecasting results with corrected group-bounded features, add transparent baselines, collect transaction-level test evidence, complete the Marketing feasibility exchange and evaluate the solution with real operational data and documented stakeholder tasks.

8. COMMERCIALISATION AND SUSTAINABILITY

8.1. Value Proposition and Target Users

The proposed users are multi-branch fashion retailers and their Inventory Managers, Merchandise Planners, Store Managers, IT Managers and operations teams. Shop-floor users, including Sales Assistants and Cashiers, may benefit indirectly from improved stock visibility, but the application has not been formally evaluated for their daily workflows. The product proposition is an integrated decision-support layer that combines demand forecasting, inventory health analysis, ranked inter-store transfer options, human-readable explanations and an approval-controlled stock movement process. It may complement existing enterprise systems rather than replace core ERP or point-of-sale functions.

Potential value includes faster access to inventory information, clearer justification of stock transfers and better coordination between inventory planning and promotional activity. These benefits are hypotheses, not measured outcomes. No production pilot, stock-out reduction, inventory-turn improvement, cost saving or return on investment was established. Commercial value would need to be tested using real before-and-after operational measures, implementation costs and a representative pilot. Informal practitioner feedback supports the relevance of the problem but cannot substitute for that evidence.

8.2. Market and Competitive Position

The initial target segment would be retailers operating several branches and a shared catalogue, where store-level stock imbalances may require coordinated transfers. Relevant alternatives include manual spreadsheet-based planning, existing ERP inventory functions and specialist forecasting or replenishment software. The proposed differentiation is the combination of cross-store transfer ranking, transparent reasons, workflow-based approval and integration with adjacent Smart Omni-Retail components. This positioning is a product hypothesis; the study contains no independently verified market-size estimate, competitor feature audit or customer

willingness-to-pay survey. Commercial claims should therefore remain conditional until such research is performed.

8.3. Business or Sustainability Model

A possible commercial approach is to offer the component as a subscription-based decision-support service or as a retailer-specific implementation and support package integrated with an existing ERP. Alternatively, the Inventory component could be licensed as part of the broader Smart Omni-Retail System. No option has been approved as a final group business model, and no subscription price, forecast revenue or break-even period is claimed. A feasibility assessment would need to measure Azure hosting, MongoDB Atlas storage and reads/writes, AI-provider usage, weather-data access, monitoring, support, model retraining and onboarding costs. Security, backup, disaster recovery, service-level commitments and customer support would also need budgeting. A commercial pilot should establish total cost of ownership before pricing.

8.4. Scalability, Intellectual Property and Compliance

The study scenario covers eight stores and approximately one thousand products, implying approximately eight thousand store-product current-state combinations if each product is represented at every store. MongoDB Atlas stores operational data, while the FastAPI backend is deployed on Azure. This architecture can be extended in principle, but the thesis provides no measured throughput, latency, concurrent-user capacity, disaster-recovery result or stress-test evidence. Production scalability would require appropriate database indexes, controlled pagination, authentication and authorization review, observability, scheduled retraining, data synchronization with retail source systems, and tests under realistic loads. The reverse Marketing campaign-feasibility integration must also be completed and verified before describing the group workflow as fully connected.

Commercial ownership of the student-created code, models and data must be determined against university policy, the group contribution agreement and any relevant industry collaboration terms before licensing or distribution. The researcher claims individual responsibility for the seven Inventory modules but not ownership of

teammates' Smart Mirror or Marketing components. Third-party library licenses, external AI service terms, weather-data terms and image usage rights require a documented review. Generated or representative product images must not be represented as licensed product photography without verification. These checks are commercialisation prerequisites, not approvals already obtained.

A commercial system would need appropriate governance for authentication, access control, audit records, data retention and secure inter-component APIs. Although the research dataset is fully synthetic, future live sales, staff or customer information may create privacy and security obligations. Any deployment in Sri Lanka should be reviewed for compliance with the applicable provisions and commencement requirements of the Personal Data Protection Act, No. 9 of 2022 [19], with legal status confirmed at deployment time. External AI-provider data handling and cross-border transfer arrangements would also need review. The present study does not establish legal or security certification.

8.5. Contribution to the Sustainable Development Goals

The system has potential, indirect relevance to the United Nations Sustainable Development Goals [20], as summarised in Table 8.1. Better inventory visibility and more deliberate transfers could support responsible resource use (SDG 12); the application of digital decision-support tools is relevant to innovation (SDG 9); and improved retail processes could contribute to productive economic activity (SDG 8). These are possible pathways, not demonstrated sustainability or economic impacts. The study did not quantify waste, carbon emissions, employment effects or resource savings.

Table 8.1: Relationship to the Sustainable Development Goals

Goal	Possible contribution
SDG 9: Industry, Innovation and Infrastructure	Applies AI and cloud tools to a practical retail operations problem in a small or mid-sized business context
SDG 12: Responsible Consumption and Production	Better matching of stock to demand could reduce overstock and the unsold inventory that results from it, and could avoid unnecessary orders when surplus stock already exists in another branch

Goal	Possible contribution
SDG 8: Decent Work and Economic Growth	Decision-support tools may help smaller retailers operate more efficiently, though this work did not study employment or economic effects

Environmental effects are not automatically positive: moving stock between branches consumes transport resources and may increase emissions if transfers are poorly consolidated. Distance and estimated logistics cost are included in the recommendation criteria, but neither emissions nor net environmental benefit was measured. A credible sustainability assessment would compare avoided overstock and replenishment against additional transfer transport using real operational data.

9. CONCLUSION AND FUTURE DIRECTIONS

9.1. Conclusion

This research designed and implemented an AI-driven Smart Inventory and Stock Flow Optimization component within the Smart Omni-Retail System. The individual implementation spans seven Inventory modules and integrates daily and monthly XGBoost-based forecasting, stock health analysis, deterministic source-store ranking, natural-language explanations, a Manager Assistant and a controlled stock movement lifecycle. The backend uses FastAPI deployed on Azure and MongoDB Atlas for persistence. The architecture separates operational decision logic from AI-generated explanations and retains human approval over movement execution. The outcome is an integrated research prototype for the stated multi-branch retail scenario, rather than a proven production solution.

The original Colab notebooks report daily MAE 0.5418, RMSE 0.9436 and R^2 0.9497, and monthly MAE 14.2128, RMSE 26.3061 and R^2 0.9353 on a chronological 2025 holdout of the team-generated synthetic dataset. These figures have not been independently reproduced after the pending grouped-feature audit and cannot establish live-retail predictive accuracy. The researcher reports successful execution of the major stock movement flows, including source and destination balance updates and transaction logging; a consolidated test-case record and authentic worked example remain pending. Informal feedback was provided by Inventory Managers, Merchandise Planners, IT Managers, Sales Assistants, Cashiers and university supervision, without a formal quantitative user study. Smart Mirror information sharing and Inventory-to-Marketing opportunities are part of the integrated implementation, whereas the reverse Marketing feasibility request and response is still partially implemented. The defensible conclusion is that the architecture and core workflows are feasible in the study setting, with further validation required before operational claims.

9.2. Achievement of Objectives

Table 9.1 maps the research objectives to the present implementation and the evidence available for this thesis revision. “Achieved” indicates that the relevant component was developed or reported functional; it does not mean that every evaluation artifact has been independently verified. Remaining evidence and integration work are identified explicitly to avoid overstating completion.

Table 9.1: Objective-achievement matrix

ID	Objective (short form)	Status	Evidence and remaining gap
O1	Python-generated five-year synthetic dataset	Achieved	Team-generated CSVs and notebooks available; generator script/rules pending documentary evidence
O2	XGBoost daily, monthly, quarterly forecasting	Largely achieved	Chronological daily/monthly holdout metrics reported; grouped rolling-feature audit, baselines and horizon-specific tests pending
O3	Inventory intelligence	Achieved	Module implemented; final documented test cases pending
O4	Deterministic recommendation engine	Achieved	Deterministic ranking implemented; worked source-ranking evidence pending
O5	Explainable AI layer	Achieved, evaluation limited	Explanatory service implemented; explanation-fidelity measurement not performed
O6	Controlled movement workflow	Achieved	Researcher reports source/destination stock updates and logs; transaction-level evidence pending
O7	Manager chatbot	Achieved, evaluation limited	Module implemented; recorded question/answer verification pending
O8	Testing and integration	Partially achieved	Smart Mirror and outbound Marketing sharing reported working; reverse campaign-feasibility exchange partially implemented; end-to-end test evidence pending

9.3. Individual Research and Product Contributions

The researcher developed the seven Inventory modules—Dashboard, Forecasting, Inventory Intelligence, Optimization, Stock Movements, Analytics and Manager Assistant—together with their supporting inventory backend services and data

integration. The individual research contribution lies in combining model-based demand estimates with transparent, deterministic transfer ranking, controlled movement execution and grounded explanation of decisions. The Python-generated synthetic dataset and group-level Smart Omni-Retail integration were collaborative activities; the Smart Mirror and Personalized Marketing Intelligence components themselves were developed by other group members. Inventory availability sharing with Smart Mirror and outbound promotional opportunities to Marketing form part of the cross-component work. The reverse Marketing-to-Inventory feasibility workflow remains partially implemented and is not claimed as an individual completed deliverable. Exact ownership and contributions should be reconciled with the official group contribution statement before submission.

9.4. Future Directions

Future directions arise from the limitations documented in Chapters 6 and 7. They include reproducible and leakage-audited forecasting experiments, real-sales validation, stronger operational testing, the completion of partially implemented cross-component connectivity and production-oriented engineering. Table 9.2 separates these next steps from functionality already present in the prototype. None of the proposed improvements is represented as an experimental result in this thesis.

Table 9.2: Future work

Direction	Purpose
Integration with real ERP and POS systems	Replace synthetic data with live stock and sales, and test the component on real operations
Real-time inventory event streaming	Keep stock records current and trigger recommendations as events occur
Larger real enterprise datasets	Test whether the reported accuracy holds on real data
Continuous model retraining	Adapt the forecasting models to changing demand
Complete bidirectional Marketing feasibility integration	Finish campaign stock-check request/response, validate availability rules, and test the full Marketing–Inventory–Marketing round trip
Richer transportation and logistics constraints, and dynamic logistics costing	Replace the simplified cost estimate and consider vehicle capacity and schedules
Documented production notifications and escalation	Assess operational alerting and escalation after reviewing the notifications already present in the prototype

Direction	Purpose
Automated manager notifications	Alert managers to shortages and pending approvals
Expanded mobile usability evaluation	Assess mobile workflows and responsive layouts beyond the current Flutter Web interface
Mobile support	Make approvals and queries available on phones
Production-scale monitoring	Track model accuracy, API quotas and system health in operation
Broader stakeholder validation	Evaluate usefulness and trust with more managers
Comparison with additional ML models	Test alternatives to XGBoost and add baselines
Feature attribution for forecasts	Add methods such as SHAP [7] to explain the forecasting model itself

REFERENCES

[1] R. Fildes, S. Ma, and S. Kolassa, "Retail forecasting: Research and practice," Int. J. Forecasting, vol. 38, no. 4, pp. 1283–1318, Dec. 2022, doi: 10.1016/j.ijforecast.2019.06.004.

[2] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "M5 accuracy competition: Results, findings, and conclusions," Int. J. Forecasting, vol. 38, no. 4, pp. 1346–1364, 2022, doi: 10.1016/j.ijforecast.2021.11.013.

[3] C. Paterson, G. Kiesmüller, R. Teunter, and K. Glazebrook, "Inventory models with lateral transshipments: A review," Eur. J. Oper. Res., vol. 210, no. 2, pp. 125–136, Apr. 2011.

[4] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD '16), San Francisco, CA, USA, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[5] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?," in Adv. Neural Inf. Process. Syst. 35 (NeurIPS 2022), Datasets and Benchmarks Track, New Orleans, LA, USA, 2022.

[6] S. Naderi, K. Kilic, and A. Dasci, "A deterministic model for the transshipment problem of a fast fashion retailer under capacity constraints," Int. J. Prod. Econ., vol. 227, 2020.

[7] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Adv. Neural Inf. Process. Syst. 30, 2017, pp. 4765–4774.

[8] A. Barredo Arrieta et al., "Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI," Inf. Fusion, vol. 58, pp. 82–115, Jun. 2020, doi: 10.1016/j.inffus.2019.12.012.

[9] Z. Ji et al., "Survey of hallucination in natural language generation," ACM Comput. Surv., vol. 55, no. 12, pp. 1–38, 2023, doi: 10.1145/3571730.

[10] MongoDB, Inc., "Databases and Collections," MongoDB Manual. [Online]. Available: https://www.mongodb.com/docs/manual/core/databases-and-collections/

[11] T. Chai and R. R. Draxler, "Root mean square error (RMSE) or mean absolute error (MAE)? – Arguments against avoiding RMSE in the literature," Geosci. Model Dev., vol. 7, no. 3, pp. 1247–1250, Jun. 2014, doi: 10.5194/gmd-7-1247-2014.

[12] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: https://otexts.com/fpp3/

[13] T. Miller, "Explanation in artificial intelligence: Insights from the social sciences," Artif. Intell., vol. 267, pp. 1–38, Feb. 2019, doi: 10.1016/j.artint.2018.07.007.

[14] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Adv. Neural Inf. Process. Syst. 33, 2020, pp. 9459–9474.

[15] A. P. Wellens, R. N. Boute, and M. Udenio, "Simplifying tree-based methods for retail sales forecasting with explanatory variables," Eur. J. Oper. Res., vol. 314, no. 2, pp. 523–539, 2024.

[16] E. A. Antipov and E. B. Pokryshevskaya, "Interpretable machine learning for demand modeling with high-dimensional data using gradient boosting machines and Shapley values," J. Revenue Pricing Manag., vol. 19, no. 5, pp. 355–364, Oct. 2020.

[17] Groq, Inc., "GroqCloud API Documentation." [Online]. Available: https://console.groq.com/docs/overview

[18] P. Zippenfenig, "Open-Meteo.com Weather API," Zenodo, 2023, doi: 10.5281/zenodo.7970649.

[19] Parliament of the Democratic Socialist Republic of Sri Lanka, "Personal Data Protection Act, No. 9 of 2022," Colombo, Sri Lanka, 2022.

[20] United Nations General Assembly, "Transforming our world: The 2030 Agenda for Sustainable Development," Resolution A/RES/70/1, Sep. 2015.

[21] K. Swaminathan and R. Venkitasubramony, “Demand forecasting for fashion products: A systematic review,” *International Journal of Forecasting*, vol. 40, no. 1, pp. 247–267, 2024, doi: 10.1016/j.ijforecast.2023.02.005.

[22] İ. Güven and F. Şimşir, “Demand forecasting with color parameter in retail apparel industry using artificial neural networks (ANN) and support vector machines (SVM) methods,” *Computers & Industrial Engineering*, vol. 147, Art. no. 106678, Sep. 2020, doi: 10.1016/j.cie.2020.106678.

[23] R. D. Reid and N. R. Sanders, *Operations Management: An Integrated Approach*, 5th ed. Hoboken, NJ, USA: Wiley, 2012, ch. 12.

[24] A. Quamar, F. Özcan, D. Miller, R. J. Moore, R. Niehus, and J. T. Kreulen, “Conversational BI: An ontology-driven conversation system for business intelligence applications,” *Proceedings of the VLDB Endowment*, vol. 13, no. 12, pp. 3369–3381, 2020, doi: 10.14778/3415478.3415557.

[25] A. B. Kanburoğlu and F. B. Tek, “Text-to-SQL: A methodical review of challenges and models,” *Turkish Journal of Electrical Engineering and Computer Sciences*, vol. 32, no. 3, pp. 403–419, 2024, doi: 10.55730/1300-0632.4077.

[26] Oracle, “Inventory Transfer Orders,” *NetSuite Applications Suite Documentation*. [Online]. Available: https://docs.oracle.com/en/cloud/saas/netsuite/ns-online-help/section_N2308766.html. [Accessed: Oct. 9, 2026].

[27] Odoo S.A., “Reordering rules,” *Odoo 18.0 Documentation*. [Online]. Available: https://www.odoo.com/documentation/18.0/applications/inventory_and_mrp/inventory/warehouses_storage/replenishment/reordering_rules.html. [Accessed: Oct. 9, 2026].

[28] Odoo S.A., “Inter-warehouse replenishment,” *Odoo 18.0 Documentation*. [Online]. Available: https://www.odoo.com/documentation/18.0/applications/inventory_and_mrp/inventory/warehouses_storage/replenishment/resupply_warehouses.html. [Accessed: Oct. 9, 2026].

[29] A. Fumi, A. Pepe, L. Scarabotti, and M. M. Schiraldi, “Fourier analysis for demand forecasting in a fashion company,” *International Journal of Engineering Business Management*, vol. 5, 2013, doi: 10.5772/56839.

GLOSSARY

Coverage percentage – The share of a target store's shortage that a proposed transfer would cover.

Days on hand – An estimate of how many days the current stock will last at the expected rate of demand.

Deterministic engine – A rule-based component that always produces the same output for the same input, with no random or learned element in its decision.

Explainable AI – Methods and design choices intended to make the behaviour or output of an AI system understandable to its users.

Groq API – A large-language-model inference service used in this project to write manager-friendly explanations of recommendations and execution summaries that the deterministic engine has already produced, and to answer the manager chatbot's open-ended questions from supplied context.

MongoDB – A document-oriented NoSQL database used in this project as the operational application database, organised into collections of documents for products, stores, current inventory, optimization candidates, stock movements, inventory transactions and cached explanations.

Open-Meteo – A free, open-source weather API called by the forecasting service at runtime to obtain current temperature, humidity and precipitation values for a store's location.

Hallucination – Text produced by a language model that is not supported by the information it was given.

Inventory transaction – A stored record of a change in stock, including the stock before and after the change.

Lag feature – A model input that holds the value of the target from an earlier period, for example demand one day or seven days ago.

Lateral transshipment – The movement of stock between locations at the same level of a supply chain, such as between two showrooms.

Overstock – A stock level that exceeds what expected demand justifies.

Reorder level – The stock quantity at which replenishment should be triggered.

Rolling statistic – A summary, such as an average, computed over a moving window of recent periods.

Safety stock – Extra stock held as a buffer against uncertainty in demand or supply.

Shortage – A stock level that is insufficient for expected demand.

Slow-moving inventory – Stock that sells at a rate well below what its quantity would suggest is appropriate.

Stock movement – A recorded transfer of a product between two stores, with a status that follows the movement lifecycle.

XGBoost – A gradient-boosted decision tree library used here for demand forecasting.

APPENDICES

The appendices contain supplementary evidence that supports the main chapters. Appendix A (Extended Results and Research Instruments) is cited from Sections 3.7, 5.4 and 6.3, and Appendix B (Academic Integrity and AI Use Disclosure) is cited from Section 3.6.

Appendix A Extended Results and Research Instruments

This appendix presents supplementary technical information and supporting evidence related to the development and evaluation of the AI-Driven Smart Inventory and Stock Flow Optimization component. The information complements the implementation and evaluation presented in Chapters 5 and 6.

A.1 Store distance data

The stock transfer recommendation engine uses store-to-store road distances and estimated transportation costs to evaluate potential inventory transfer options.

The research dataset contains 28 active distance records representing unique store pairs within an eight-store retail network. These records are maintained in the MongoDB Atlas store_distances collection and include road distance, estimated travel time, transportation cost, and verification information.

The records identify Google Maps as the verification source, with a recorded verification date of 7 July 2026. The estimated transportation cost is calculated using a configured rate of LKR 200 per kilometre.

To maintain confidentiality, the stores are identified as Store 01 through Store 08. The existing store identifiers (CP001–CP008) are retained for consistency with the research dataset and application implementation.

Table A.1: Store-to-store distances used by the engine

From Store	To Store	Distance (km)	Travel Time (min)	Estimated Cost (LKR)
CP001	CP002	4.7	14	940
CP001	CP003	123.0	197	24,600
CP001	CP004	3.7	11	740
CP001	CP005	18.7	47	3,740
CP001	CP006	12.9	32	2,580
CP001	CP007	7.2	22	1,440
CP001	CP008	13.5	34	2,700
CP002	CP003	122.0	195	24,400
CP002	CP004	7.9	24	1,580
CP002	CP005	37.8	60	7,560
CP002	CP006	15.0	38	3,000
CP002	CP007	7.0	21	1,400
CP002	CP008	12.6	32	2,520

From Store	To Store	Distance (km)	Travel Time (min)	Estimated Cost (LKR)
CP003	CP004	124.0	198	24,800
CP003	CP005	113.0	181	22,600
CP003	CP006	124.0	198	24,800
CP003	CP007	120.0	192	24,000
CP003	CP008	110.0	176	22,000
CP004	CP005	12.8	32	2,560
CP004	CP006	7.9	24	1,580
CP004	CP007	8.6	26	1,720
CP004	CP008	15.4	38	3,080
CP005	CP006	11.2	28	2,240
CP005	CP007	15.8	40	3,160
CP005	CP008	16.6	42	3,320
CP006	CP007	12.5	31	2,500
CP006	CP008	20.4	51	4,080
CP007	CP008	8.1	24	1,620

Source: MongoDB Atlas store_distances collection. The records identify Google Maps as the verification source, with a last-verification date of 7 July 2026.

Store ID	Store Name
CP001	Store 01
CP002	Store 02
CP003	Store 03
CP004	Store 04
CP005	Store 05
CP006	Store 06
CP007	Store 07
CP008	Store 08

The estimated transfer cost is calculated using the following formula:

Estimated Transfer Cost (LKR) = Road Distance (km) × Transportation Cost per km (LKR)

For example, the recorded distance between Store 01 (CP001) and Store 02 (CP002) is 4.7 km. Applying the configured transportation rate of LKR 200 per kilometre produces an estimated transfer cost of LKR 940.

The recorded road distances range from 3.7 km to 124.0 km. These differences allow the recommendation engine to consider geographical separation and estimated transportation costs when comparing potential stock transfer options.

The distance and cost information provides supporting inputs for the optimization process. However, the recorded travel times and transportation costs are estimates and may differ from actual conditions due to traffic, vehicle availability, transportation arrangements, and other operational factors.

The dataset contains one record for each unique store pair. The presence of a stored pair does not, by itself, establish whether the application treats travel distances as identical in both directions; that behaviour depends on the recommendation engine's lookup implementation.

For confidentiality and research ethics considerations, the retail store names have been anonymized as Store 01 through Store 08. The original store identifiers (CP001–CP008) have been retained to maintain consistency across the synthetic dataset, database records, forecasting models, and stock transfer optimization processes. The research findings are based on a simulated retail environment and do not represent the actual operational performance of any identified retail organization.

A.2 Full test-case record

Functional testing was conducted to verify the major workflows of the Smart Inventory application. The testing activities included demand forecasting, inventory intelligence, stock transfer recommendation, movement approval, execution, inventory synchronization, and transaction recording.

The researcher reported successful execution of the principal inventory workflows, including changes to source and destination inventory quantities following an approved stock transfer.

This subsection provides selected supporting evidence for the functional testing activities described in Section 5.4 and the results discussed in Section 6.2.

Table A.2: Detailed test-case record

ID	Test Area	Verification Scenario
TC01	Forecasting	Generation of daily and monthly demand predictions
TC02	Inventory Intelligence	Identification of inventory health and shortage conditions
TC03	Stock Optimization	Generation of recommended source stores and alternatives
TC04	Stock Movement	Approval and execution of an inventory transfer

ID	Test Area	Verification Scenario
TC05	Inventory Synchronization	Updating source and destination stock quantities
TC06	Transaction History	Recording completed stock movement transactions

A.3 Model Configuration and Technical Specifications

The demand forecasting component uses separately trained XGBoost regression models for daily and monthly forecasting. Quarterly forecasts are generated through the aggregation of three sequential monthly predictions.

The backend application was developed using FastAPI and deployed on Microsoft Azure. MongoDB Atlas provides operational data storage, while Firebase Authentication supports user authentication.

Additional external services include the Groq API for AI-generated explanations and conversational assistance, and Open-Meteo for weather-related information.

A.4 Selected Technical Specifications

Table A.3: Selected Technical Specifications

Component	Technology or Configuration
Frontend	Flutter Web (Dart)
Backend	Python (FastAPI)
Operational Database	MongoDB Atlas
Authentication	Firebase Authentication
Machine Learning	XGBoost Regressor
Forecasting Models	Daily and Monthly
Quarterly Forecasting	Aggregation of three sequential monthly forecasts
Explainable AI	Groq API
Weather Data	Open-Meteo API
Backend Deployment	Microsoft Azure
Source Code Management	Git and GitHub

Appendix B Academic Integrity and AI Use Disclosure

This appendix documents the use of generative artificial intelligence tools during the development and preparation of the research thesis.

Generative AI was used for supporting activities such as document structuring, academic writing improvement, technical review, and development assistance. These activities are distinguished from the AI functionality integrated into the Smart Inventory application itself.

The Groq API was incorporated into the developed system to generate explanations and support conversational responses. Its use as part of the application is described in the implementation chapters.

Table B.1: AI use disclosure

AI tool and version	Purpose	Representative Use	Verification & Responsibility
Claude (Anthropic)	Initial thesis drafting and document structuring	Preparing an initial draft using the thesis template, research information, and formatting requirements	Technical descriptions, references, and reported findings to be checked against original project records
ChatGPT (OpenAI)	Academic writing improvement and technical review	Revising thesis sections, improving academic language, organizing tables, and identifying missing evidence	AI-assisted material to be reviewed against actual implementation, research records, and supporting sources

B.1 Declaration statement

Generative AI tools were used to support the preparation and refinement of this research thesis, including academic writing, document organization, language improvement, and technical review.

The research objectives, system implementation, dataset generation, machine learning experiments, and reported findings remain the responsibility of the researcher. AI-assisted material will be reviewed and verified against the original project implementation, research records, and supporting references before final submission.

The researcher accepts full responsibility for the accuracy, originality, academic integrity, and final content of the submitted thesis.